

PlatEMO

Evolutionary Multi-Objective Optimization Platform

User Manual 4.17

BIMK Group

October 31, 2026

Thank you very much for using PlatEMO. The copyright of PlatEMO belongs to the BIMK Group of Anhui University, China. This platform is only for research and educational purposes. The codes were implemented based on our understanding of the algorithms published in literatures. You should not rely upon the material or information provided by the platform as a basis for making any business, legal or any other decisions. We assume no responsibilities for any consequences of your using any codes in the platform. All publications using the platform should acknowledge the use of "PlatEMO" and cite one of the following two papers:

[1] Ye Tian, Weijian Zhu, Xingyi Zhang, and Yaochu Jin, "A practical tutorial on solving optimization problems via PlatEMO," Neurocomputing, 2023, 518: 190-205.

[2] Ye Tian, Ran Cheng, Xingyi Zhang, and Yaochu Jin, "PlatEMO: A MATLAB platform for evolutionary multi-objective optimization [educational forum]," IEEE Computational Intelligence Magazine, 2017, 12(4): 73-87.

If you have any comment or suggestion to PlatEMO, please send it to *field910921@gmail.com* (*Prof. Ye Tian*). If you want to add your code to PlatEMO, please send the ready-to-use code and the relevant literature to *field910921@gmail.com* as well. You can obtain the newest version of PlatEMO from GitHub.

Contents

I. Quick Start 1

II. Using PlatEMO without GUI 3

A. Solving Benchmark Problems 3

B. Solving User-Defined Problems 5

C. Collecting the Results 10

III. Using PlatEMO with GUI 12

A. Test Module 12

B. Application Module 13

C. Experiment Module 14

D. Creation Module 15

E. Labels of Algorithms, Problems, and Metrics 16

IV. Extending PlatEMO 18

A. ALGORITHM Class 18

B. PROBLEM Class 20

C. SOLUTION Class 26

D. Whole Procedure of One Run 27

E. Metric Function 28

F. Creating NeuroEAs 29

V. List of Algorithms 37

VI. List of Problems 51

I. Quick Start

Requirement: MATLAB R2018a or higher (PlatEMO without GUI) or
MATLAB R2020b or higher (PlatEMO with GUI) with
Parallel Computing Toolbox and
Statistics and Machine Learning Toolbox

PlatEMO is an open-source platform for solving optimization problems, whose input is an optimization problem and output is the found optimal solutions. An optimization problem is defined as

$$\begin{aligned}\min_{\mathbf{x}} \quad & \mathbf{f}(\mathbf{x}) = \left(f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x})\right) \\ \text{s.t.} \quad & \mathbf{x} = (x_1, x_2, \dots, x_D) \in \Omega \\ & g_1(\mathbf{x}), g_2(\mathbf{x}), \dots, g_K(\mathbf{x}) \leq 0\end{aligned}$$

where **x** denotes a **solution** or **decision vector** for the problem, which consists of D **decision variables** x_i, and each decision variable can be a real number, integer, binary number, or others. Ω denotes the **search space** of the problems, which consists of the **lower bounds** $l_1, l_2, \dots, l_D$ and the **upper bounds** $u_1, u_2, \dots, u_D$, i.e., each decision variable should always satisfy that $l_i \leq x_i \leq u_i$. $f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x})$ denote the M **objective values** of the solution, and $g_1(\mathbf{x}), g_2(\mathbf{x}), \dots, g_K(\mathbf{x})$ denote the K **constraint violations** of the solution.

To define an optimization problem, users should input at least the following contents:

- The encoding scheme of each decision variable (real, integer, binary, etc.);
- The lower bounds $l_1, l_2, \dots, l_D$ and the upper bounds $u_1, u_2, \dots, u_D$;
- At least one objective function $f_1(\mathbf{x})$.

To define an optimization problem more precisely, users can also input the following contents:

- Multiple objective functions $f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x})$;
- Multiple constraint functions $g_1(\mathbf{x}), g_2(\mathbf{x}), \dots, g_K(\mathbf{x})$;
- Function for initializing solutions;
- Function for repairing invalid solutions;
- Function for evaluating solutions;
- Function for calculating the gradients of objectives and constraints;
- Data used in the calculation of all functions (an arbitrary constant).

The above functions are MATLAB functions rather than mathematical functions, which should have specified inputs and outputs but need not have explicit mathematical

expressions. Moreover, users can define the settings of optimization algorithms, to achieve the improvement of optimization performance via selecting suitable algorithms and parameter settings.

In MATLAB, users can call the main file `platemo.m` in the following three ways:

1) Calling the main function with parameters:

```
platemo('problem',@SOP_F1,'algorithm',@GA);
```

Then the specified benchmark problem will be solved by the specified algorithm with specified parameter settings, where the result can be displayed, saved, or returned (see *Solving Benchmark Problems* for details).

2) Calling the main function with parameters:

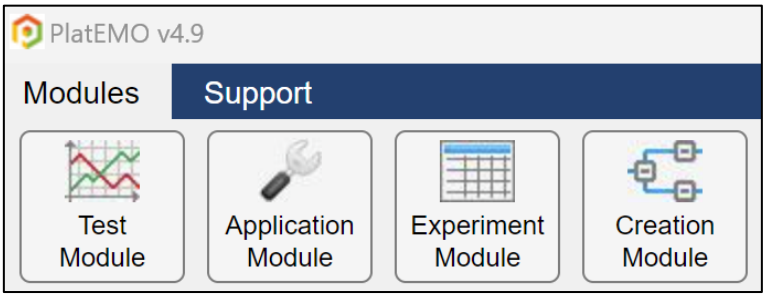
```
f1 = @(x)sum(x);
f2 = @(x)1-sum(x);
platemo('objFcn',f1,'conFcn',f2,'algorithm',@GA);
```

Then the user-defined problem will be solved by the specified algorithm with specified parameter settings (see *Solving User-Defined Problems* for details).

3) Calling the main function without parameter:

```
platemo();
```

Then a GUI with four modules will be displayed, where the test module is used to visually investigate the performance of an algorithm on a benchmark problem (see *Functions of Test Module* for details), the application module is used to solve user-defined problems (see *Functions of Application Module* for details), the experiment module is used to statistically analyze the performance of multiple algorithms on multiple benchmark problems (see *Functions of Experiment Module* for details), and the creation module is used to create new algorithms (NeuroEAs) without writing code (see *Functions of Creation Module* for details).



II. Using PlatEMO without GUI

A. Solving Benchmark Problems

Users can use PlatEMO without GUI by calling the main function `platemo()` with parameters like

```
platemo('Name1',Value1,'Name2',Value2,'Name3',Value3);
```

where all the acceptable names and values are

Name	Data type	Default value	Description
`'algorithm'`	Function handle or cell	`dependent`	Class of algorithm
`'problem'`	Function handle or cell	`dependent`	Class of problem
`'N'`	Positive integer	`100`	Population size
`'M'`	Positive integer	`dependent`	Number of objectives
`'D'`	Positive integer	`dependent`	Number of variables
`'maxFE'`	Positive integer	`10000`	Maximum number of function evaluations
`'maxRuntime'`	Positive number	`inf`	Maximum runtime
`'save'`	Integer	`-10`	Number of saved populations
`'run'`	Positive integer	`[]`	Current execution number
`'metName'`	Function handle or cell	`{}`	Names of metrics to calculate
`'outputFcn'`	Function handle	`@DefaultOutput`	Function called before each iteration Input 1: Class of algorithm Input 2: Class of problem Output: None

- `'algorithm'` denotes the algorithm to be run, whose value should be the function handle of an algorithm, such as `@GA`. The value can also be a cell like `{@GA,p1,p2,…}`, where `p1,p2,…` specify the parameter values of the algorithm. For example, the following code solves the default problem via the algorithm `@GA` with specified parameters:

```
platemo('algorithm',{@GA,1,30,1,30});
```

- `'problem'` denotes the benchmark problem to solve, whose value should be the function handle of a benchmark problem, such as `@SOP_F1`. The value can also be a cell like `{@SOP_F1,p1,p2,…}`, where `p1,p2,…` specify the parameter values

of the benchmark problem. For example, the following code solves the problem `@WFG1` with specified parameters via the default algorithm:

```
platemo('problem',{@WFG1,20});
```

- `'N'` denotes the population size of the algorithm, which usually equals the number of solutions in the final population. For example, the following code solves the problem `@SOP_F1` via the algorithm `@GA` with a population size of 50:

```
platemo('algorithm',@GA,'problem',@SOP_F1,'N',50);
```

- `'M'` denotes the number of objectives of the benchmark problem, which is valid for some multi-objective benchmark problems. For example, the following code solves the problem `@DTLZ2` with 5 objectives via the algorithm `@NSGAII`:

```
platemo('algorithm',@NSGAII,'problem',@DTLZ2,'M',5);
```

- `'D'` denotes the number of decision variables of the benchmark problem, which is valid for some benchmark problems. For example, the following code solves the problem `@SOP_F1` with 100 variables via the algorithm `@GA`:

```
platemo('algorithm',@GA,'problem',@SOP_F1,'D',100);
```

- `'maxFE'` denotes the maximum number of available function evaluations, which usually equals the product of population size and number of generations. For example, the following code sets the maximum number of function evaluations to 20000 for the algorithm `@GA`:

```
platemo('algorithm',@GA,'problem',@SOP_F1,'maxFE',20000);
```

- `'maxRuntime'` denotes the maximum runtime (in second). When `'maxRuntime'` equals its default value `inf`, the algorithm will terminate after `'maxFE'` function evaluations; otherwise, the algorithm will terminate after `'maxRuntime'` seconds. For example, the following code sets the maximum runtime to 10 seconds for the algorithm:

```
platemo('algorithm',@GA,'problem',@SOP_F1,'maxRuntime',10);
```

- `'save'` denotes the number of saved populations, where the populations are saved to a file if the value is positive and displayed in a figure if the value is negative (see *Collecting the Results* for details).
- `'run'` denotes the current execution number, which is involved in the name of saved files, differentiating the names of files saved for multiple executions of an algorithm on a problem (see *Collecting the Results* for details).
- `'metName'` denotes the names of metrics to calculate, whose value can be a string (a single metric) or a cell (multiple metrics). The metric values of saved

populations are calculated, and then are saved to a file or displayed in a figure (see *Collecting the Results* for details).

- `'outputFcn'` denotes the function called before each iteration of the algorithm. An output function has two inputs and no output, where the first input is the current `ALGORITHM` object and the second input is the current `PROBLEM` object. The default `'outputFcn'` saves or displays the populations according the value of `'save'`.

Note that users need not specify all the parameters as each of them has a default value.

B. Solving User-Defined Problems

When the parameter `'problem'` is not specified, users can define their own problems by specifying the following parameters:

Name	Data type	Default value	Description
`'objFcn'`	Function handle, matrix, or cell	`{}`	Objective functions; all the objectives are to be minimized Input: A decision vector Output: Objective value (scalar)
`'encoding'`	Scalar or row vector	`1`	Encoding scheme of each variable
`'lower'`	Scalar or row vector	`0`	Lower bound of each variable
`'upper'`	Scalar or row vector	`1`	Upper bound of each variable
`'conFcn'`	Function handle, matrix, or cell	`{}`	Constraint functions; a constraint is satisfied if and only if the constraint violation is not positive Input: A decision vector Output: Constraint violation (scalar)
`'decFcn'`	Function handle	`{}`	Function for repairing an invalid solution Input: A decision vector Output: Repaired decision vector
`'evalFcn'`	Function handle	`{}`	Function for evaluating a solution Input: A decision vector Output 1: Repaired decision vector Output 2: All objective values (vector) Output 3: All constraint violations (vector)
`'initFcn'`	Function handle	`{}`	Function for initializing a population Input: Population size Output: A matrix consisting of the decision vectors of all solutions
`'gradFcn'`	Function handle	`{}`	Function for calculating the gradients of a solution on objectives and constraints Input: A decision vector Output 1: Jacobian matrix of objectives Output 2:Jacobian matrix of constraints
`'data'`	Any	`{}`	Data of the problem

'once'	Logical	0	Whether multiple solutions can be evaluated simultaneously

- 'objFcn' denotes the objective functions of the problem, whose value can be a function handle (a single objective), a matrix (a function is automatically fitted), or a cell (multiple objectives). An objective function has one input and one output, where the input is a decision vector and the output is the objective value. All the objectives are to be minimized. For example, the following code solves a bi-objective optimization problem with six real variables via the default algorithm:

```
f1 = @(x)x(1)+sum(x(2:end));
f2 = @(x)sqrt(1-x(1)^2)+sum(x(2:end));
platemo('objFcn',{f1,f2},'D',6);
```

  where the first objective is $x_1 + \sum_{i=2}^{D} x_i$ and the second objective is $\sqrt{1-x_1^2} + \sum_{i=2}^{D} x_i$. If an objective function is a matrix, a function will be automatically fitted via Gaussian process regression, where each row of the matrix is a sample and each column of the matrix is a variable (except for the last column) or a function value (the last column). For example, the following code solves the same problem, while the objective functions are automatically fitted:

```
x = rand(50,6);
y1 = x(:,1)+sum(x(:,2:end),2);
y2 = sqrt(1-x(:,1).^2)+sum(x(:,2:end),2);
platemo('objFcn',{[x,y1],[x,y2]},'D',6);
```

- 'encoding' denotes the encoding scheme of each variable, whose value can be a scalar or row vector, and the value of each dimension can be 1 (real number), 2 (integer), 3 (label), 4 (binary number), or 5 (permutation number). The algorithms may generate solutions via different strategies for different encoding schemes. For example, the following code specifies three real variables, two integer variables, and one binary variable:

```
f1 = @(x)x(1)+sum(x(2:end));
f2 = @(x)sqrt(1-x(1)^2)+sum(x(2:end));
platemo('objFcn',{f1,f2},'encoding',[1,1,1,2,2,4]);
```

  the number of variables *D* is automatically set to the length of 'encoding'.

- 'lower' and 'upper' denote the lower and upper bound of each variable, respectively, whose values can be scalars or row vectors, and the value of each dimension should be real. 'lower' and 'upper' should have the same length as 'encoding'. For example, the following code specifies a search space $[0,1] \times [0,9]^5$:

```
f1 = @(x)x(1)+sum(x(2:end));
f2 = @(x)sqrt(1-x(1)^2)+sum(x(2:end));
platemo('objFcn',{f1,f2},'encoding',[1,1,1,2,2,4],...
'lower',0,'upper',[1,9,9,9,9,9]);
```

- `'conFcn'` denotes the constraint functions of the problem, whose value can be a function handle (a single constraint), a matrix (a function is automatically fitted), or a cell (multiple constraints). A constraint function has one input and one output, where the input is a decision vector and the output is the constraint violation. A constraint is satisfied if and only if the constraint violation is not positive. For example, the following code solves a bi-objective optimization problem via the default algorithm:

```
f1 = @(x)x(1)+sum(x(2:end));
f2 = @(x)sqrt(1-x(1)^2)+sum(x(2:end));
g1 = @(x)1-sum(x(2:end));
platemo('objFcn',{f1,f2},'encoding',[1,1,1,2,2,4],...
'conFcn',g1,'lower',0,'upper',[1,9,9,9,9,9]);
```

  and adds a constraint $\sum_{i=2}^{6} x_i \geq 1$. Note that equality constraints should be converted into inequality constraints, the details of which can be found in Section 3.2 of *this paper*. If a constraint function is a matrix, a function will be automatically fitted via Gaussian process regression, where each row of the matrix is a sample and each column of the matrix is a variable (except for the last column) or a function value (the last column). For example, the following code solves the same problem, while the constraint function is automatically fitted:

```
f1 = @(x)x(1)+sum(x(2:end));
f2 = @(x)sqrt(1-x(1)^2)+sum(x(2:end));
x = rand(50,6);
y = 1-sum(x(:,2:end),2);
platemo('objFcn',{f1,f2},'encoding',[1,1,1,2,2,4],...
'conFcn',[x,y],'lower',0,'upper',[1,9,9,9,9,9]);
```

- `'decFcn'` denotes the function for repairing an invalid solution, whose value should be a function handle having one input and one output, where the input is a decision vector and the output is the repaired decision vector. The default `'decFcn'` limits each solution within the search space determined by `'lower'` and `'upper'`, while the following code defines a new `'decFcn'` to make x_1 always be a multiple of 0.1:

```
f1 = @(x)x(1)+sum(x(2:end));
f2 = @(x)sqrt(1-x(1)^2)+sum(x(2:end));
g1 = @(x)1-sum(x(2:end));
```

```
h = @(x)[round(x(1)/0.1)*0.1,x(2:end)];
platemo('objFcn',{f1,f2},'encoding',[1,1,1,2,2,4],...
'conFcn',g1,'decFcn',h,'lower',0,'upper',[1,9,9,9,9,9]);
```

- `'evalFcn'` denotes the function for evaluating a solution, whose value should be a function handle having one input and three output, where the input is a decision vector, the first output is the repaired decision vector, the second output is the vector of objective values, and the third vector is the vector of constraint violations. The default `'evalFcn'` calls `'decFcn'`, `'objFcn'`, and `'conFcn'` in sequence to evaluate a solution, while the following code defines a new `'evalFcn'` to achieve solution repair, objective calculation, and constraint calculation:

```
function [x,f,g] = Eval(x)
    x = [round(x(1)/0.1)*0.1,x(2:end)];
    x = max(0,min([1,9,9,9,9,9],x));
    f(1) = x(1)+sum(x(2:end));
    f(2) = sqrt(1-x(1)^2)+sum(x(2:end));
    g = 1-sum(x(2:end));
end
```

  Then, the following codes solve the same problem by specifying only the evaluation function:

```
platemo('evalFcn',@Eval,'encoding',[1,1,1,2,2,4],...
'lower',0,'upper',[1,9,9,9,9,9]);
```

- `'initFcn'` denotes the function for initializing a population, whose value should be a function handle having one input and one output, where the input is the number of solutions in the population and the output is a matrix consisting of the decision vectors in the population. The default `'initFcn'` randomly generates solutions in the whole search space, while the following code defines a new `'initFcn'` to accelerate the convergence:

```
q = @(N)rand(N,6);
platemo('evalFcn',@Eval,'encoding',[1,1,1,2,2,4],...
'initFcn',q,'lower',0,'upper',[1,9,9,9,9,9]);
```

- `'gradFcn'` denotes the function for calculating the gradients of a solution on objectives and constraints, whose value should be a function handle having one input and two outputs, where the input is a decision vector, the first output is the Jacobian matrix of objectives, and the second output is the Jacobian matrix of constraints. The default gradient function estimates the gradients via finite difference, while the following code defines a new `'objGradFcn'`:

```
function [oGrad,cGrad] = Grad(x)
```

```
    oGrad = [0,x(2:end);0,x(2:end)];
    cGrad = [0,x(2:end)-1/5];
end
```

Then, the following codes specify the gradient function:

```
platemo('evalFcn',@Eval,'encoding',[1,1,1,2,2,4],...
'gradFcn',@Grad,'lower',0,'upper',[1,9,9,9,9,9]);
```

Note that only a few algorithms use gradient functions.

- `'data'` denotes the data of the problem, which can be a constant of any type. If `'data'` is specified, all the above functions should have an additional input to receive `'data'`. For example, the following code solves a rotated single-objective optimization problem:

```
d = rand(RandStream('mlfg6331_64','Seed',28),10)*2-1;
[d,~] = qr(d);
f1 = @(x,d)sum((x*d-0.5).^2);
platemo('objFcn',f1,'encoding',ones(1,10),'data',d);
```

- `'once'` indicates whether multiple solutions can be evaluated simultaneously, which should be a logical variable with default value of zero. When the value of `'once'` is set to 1, the inputs of `'evalFcn'`, `'decFcn'`, `'objFcn'`, and `'conFcn'` can be multiple decision vectors, i.e., evaluating multiple solutions simultaneously. Using matrix calculation or parallel calculation in functions can significantly improve the efficiency. For example, the following code updates the objective function with matrix calculation:

```
d = rand(RandStream('mlfg6331_64','Seed',28),10)*2-1;
[d,~] = qr(d);
f1 = @(x,d)sum((x*d-0.5).^2,2);
platemo('objFcn',f1,'encoding',ones(1,10),'data',d,'once',1);
```

In addition to the above way for defining a problem, a problem object can be created and solved by specified algorithm objects. For example, the following code solves the problem via the algorithm `@GA` and the algorithm `@DE`.

```
d = rand(RandStream('mlfg6331_64','Seed',28),10)*2-1;
[d,~] = qr(d);
f1 = @(x,d)sum((x*d-0.5).^2);
PRO = UserProblem('objFcn',f1,'encoding',ones(1,10),'data',d);
ALG1 = GA();
ALG2 = DE();
ALG1.Solve(PRO);
ALG2.Solve(PRO);
```

C. Collecting the Results

The generated populations can be displayed, saved, or returned after the algorithm terminates. If the main function is called like

```
[Dec,Obj,Con] = platemo('Name1',Value1,'Name2',Value2);
```

Then the final population will be returned, where `Dec` is a matrix consisting of the decision vectors in the final population, `Obj` is a matrix consisting of the objective values in the final population, and `Con` is a matrix consisting of the constraint violations in the final population. If the main function is called like

```
platemo('save',Value);
```

Then the generated populations will be displayed in a figure if `Value` is negative (default), where various plots can be displayed by switching the `Data source` menu on the figure. While if `Value` is positive, the generated populations will be saved to a MAT file named as `PlatEMO\Data\alg\alg_pro_M_D_run.mat`, where `alg` is the algorithm name, `pro` is the problem name, `M` is the number of objectives, `D` is the number of variables, and `run` automatically increases from 1 until the file name does not exist. Moreover, the value of `run` can be explicitly specified by

```
parfor i = 1 : 100
    platemo('save',Value,'run',i);
end
```

where `run` increases from 1 to 100. When multiple runs are performed in parallel, specifying the value of `run` can avoid the confusion or missing of file numbers.

Each file saves a cell `result` consisting of the generated populations and a struct `metric` consisting of the metric values. The whole optimization process of the algorithm is divided into `Value` equal intervals, where the first column of `result` stores the number of consumed function evaluations at the last iteration of each interval, the second column of `result` stores the population at the last iteration of each interval, and `metric` stores the metric values of the stored populations.

```
result =
  6×2 cell array
    {[ 1600]}    {1×100 SOLUTION}
    {[ 3300]}    {1×100 SOLUTION}
    {[ 5000]}    {1×100 SOLUTION}
    {[ 6600]}    {1×100 SOLUTION}
    {[ 8300]}    {1×100 SOLUTION}
    {[10000]}    {1×100 SOLUTION}
```

```
metric =
  struct with fields:

    runtime: 0.2267
        IGD: [6×1 double]
         HV: [6×1 double]
```

Setting the parameter `'metName'` to specify the metrics to calculate, for example, the

following code solves the problem `@DTLZ2` via the algorithm `@NSGAII` and saves the metric values of IGD and HV to a file:

```
platemo('algorithm',@NSGAII,'problem',@DTLZ2,...
'save',6,'metName',{'IGD','HV'});
```

where `'IGD'` and `'HV'` are the names of the metrics to calculate (see *Metric Function* for details). In particular, IGD and HV are the most popular metrics for multi-objective optimization, whose application scopes and methods for defining reference points can be found in Section 5.3 of *this paper*. The above are achieved by the default output function `@DefaultOutput`, while users can collect the results in their own ways by specifying the value of `'outputFcn'` to the handle of a user-defined output function. Besides, the metric value of a single population can be calculated by

```
% Load result before performing the following code
pro = DTLZ2();
pro.CalMetric('IGD',result{end});
```

Also, the metric values can be automatically calculated and saved in the experiment module of the GUI.

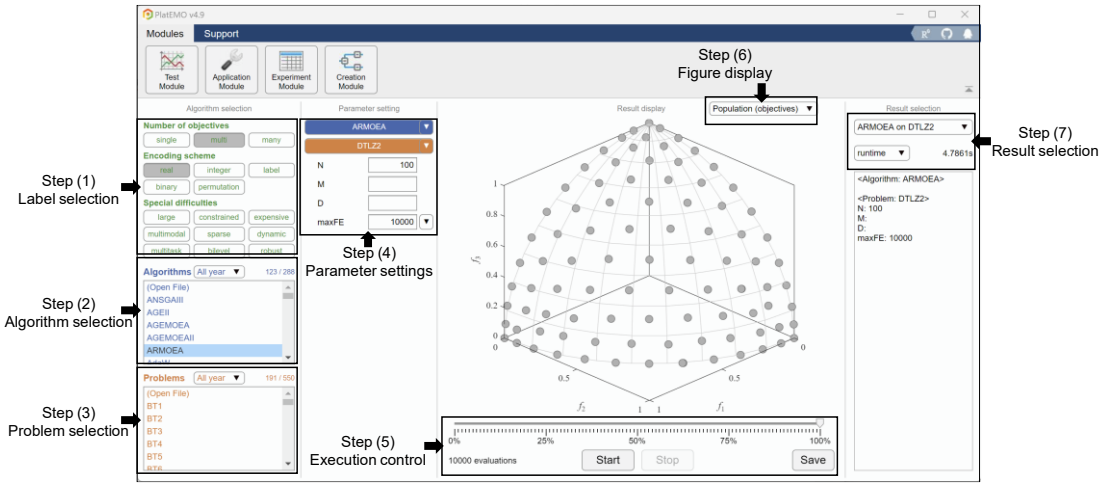
III. Using PlatEMO with GUI

A. Test Module

Users can use PlatEMO with GUI by calling the main function `platemo()` without parameter like

```
platemo();
```

Then the test module of the GUI will be displayed, which is used to visually investigate the performance of an algorithm on a benchmark problem.

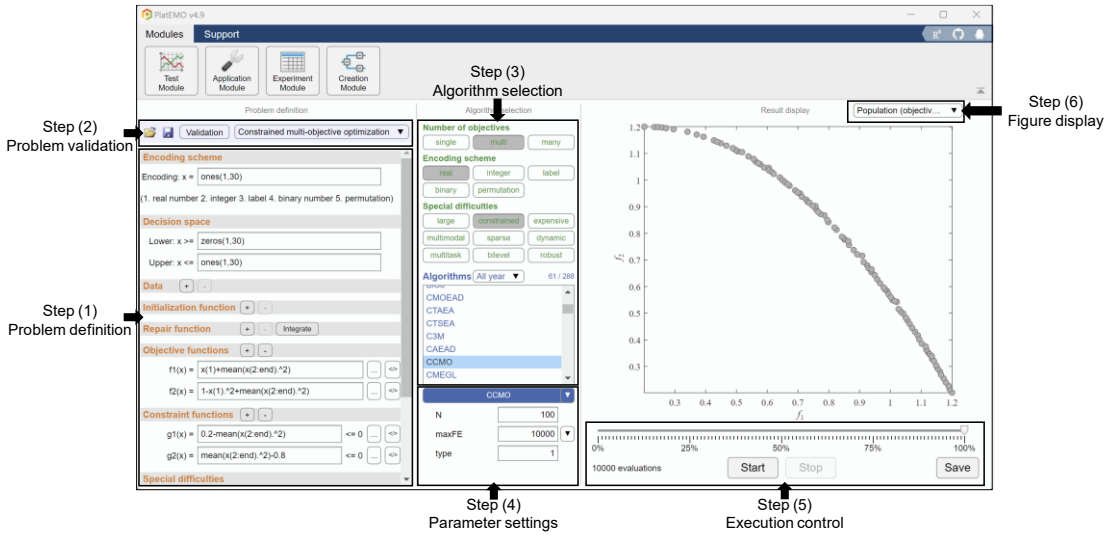


In this module, the performance investigation can be achieved by the following steps:

- Step (1) Select multiple labels to determine the type of problems (see *Labels of Algorithms, Problems, and Metrics* for details).
- Step (2) Select an algorithm from the list.
- Step (3) Select a benchmark problem from the list.
- Step (4) Set the parameters of the algorithm and benchmark problem. Different algorithms and benchmark problems may have different parameters, the details of which can be obtained by hovering over each parameter.
- Step (5) Start, pause, stop, or back off the current execution; save the current result to a file. The current result can be saved as a matrix with N rows and $D + M + K$ columns, where N denotes the number of solutions, D denotes the number of variables, M denotes the number of objectives, and K denotes the number of constraints.
- Step (6) Select a data to display, such as the objective values, variables, and metric values of the current population.
- Step (7) Select a historical result to display.

B. Application Module

Users can press the menu button to switch to the application module, which is used to solve user-defined problems.

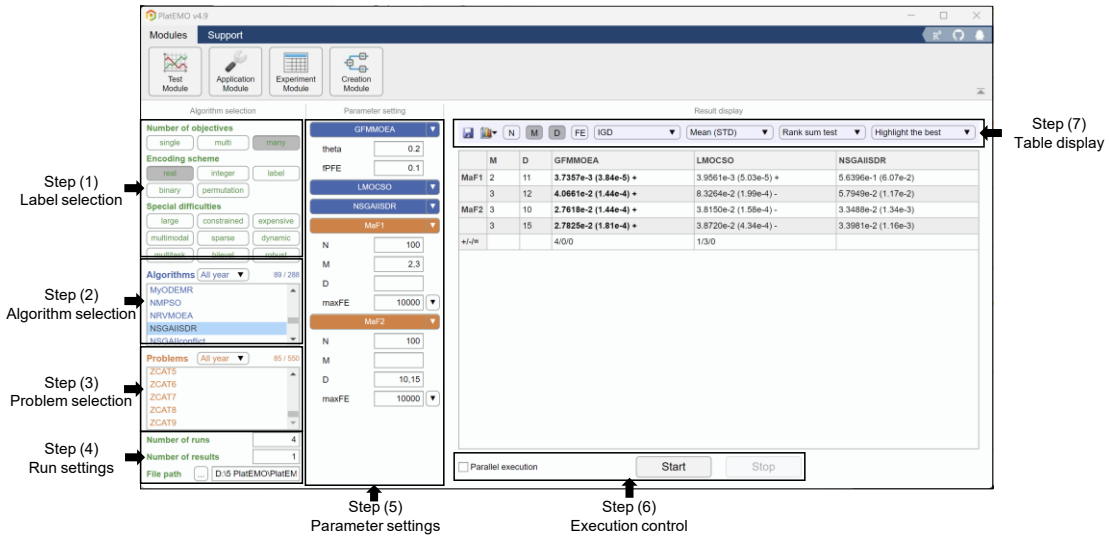


In this module, the solving of problems can be achieved by the following steps:

- Step (1) Define a problem, the contents of which are the same as those in *Solving User-Defined Problems*, where Encoding scheme corresponds to `'encoding'`, Decision space corresponds to `'lower'` and `'upper'`, Data corresponds to `'data'`, Initialization function corresponds to `'initFcn'`, Repair function corresponds to `'decFcn'`, Objective functions corresponds to `'objFcn'`, Constraint functions corresponds to `'conFcn'`, and Evaluation function corresponds to `'evalFcn'`.
- Step (2) Save or load a problem; check the validity of the problem; select a problem template. The saved problem can be opened and solved in other modules.
- Step (3) Select an algorithm from the list. The labels are automatically determined according to the problem definition (see *Labels of Algorithms, Problems, and Metrics* for details).
- Step (4) Set the parameters of the algorithm. Different algorithms may have different parameters, the details of which can be obtained by hovering over each parameter.
- Step (5) Start, pause, stop, or back off the current execution; save the current result to a file. The current result can be saved as a matrix with N rows and $D + M + K$ columns, where N denotes the number of solutions, D denotes the number of variables, M denotes the number of objectives, and K denotes the number of constraints.
- Step (6) Select a data to display, such as the objective values, variables, and metric values of the current population.

C. Experiment Module

Users can press the menu button to switch to the experiment module, which is used to statistically analyze the performance of multiple algorithms on multiple problems. The results generated in this module will be saved to MAT files (see *Collecting the Results* for details), and results will be loaded from existing files without execution.

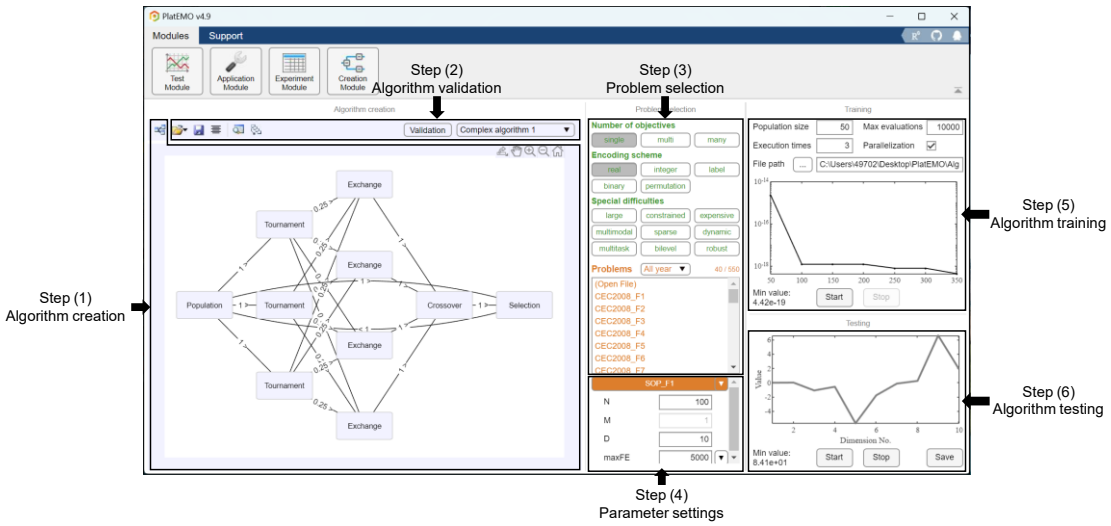


In this module, comparative experiments can be achieved by the following steps:

- Step (1) Select multiple labels to determine the type of problems (see *Labels of Algorithms, Problems, and Metrics* for details).
- Step (2) Select multiple algorithms from the list.
- Step (3) Select multiple benchmark problems from the list.
- Step (4) Set the number of repeated runs, number of saved populations in each run, and path for saving results (see *Collecting the Results* for details).
- Step (5) Set the parameters of the algorithms and benchmark problems. Different algorithms and benchmark problems may have different parameters, the details of which can be obtained by hovering over each parameter. Here the parameters of benchmark problems can be vectors, which generates multiple test instances based on a single benchmark problem.
- Step (6) Start or stop the experiment; perform multiple runs in sequence (on a single CPU) or in parallel (on all CPUs).
- Step (7) Select a metric; select a statistical method; save the table to a file; display the results of the selected cells in a figure.

D. Creation Module

Users can press the menu button to switch to the creation module, which is used to create totally new algorithms (NeuroEAs) and train them on benchmark problems. Details about NeuroEAs can be found in *this paper*, and the way of creating NeuroEAs without GUI is referred to *Creating NeuroEAs*.



In this module, new algorithms can be created and trained by the following steps:

- Step (1) Add new blocks by clicking on the button, add new connections by clicking on two blocks, change the layout by dragging blocks and connections. Blocks include population block, operator blocks, and selection blocks, where each block has some predefined hyperparameters and some parameters to train; connections indicate the transmission directions and ratios of solutions between blocks. An algorithm is regarded as a directed and weighted cyclic graph with nodes of blocks and edges of connections, where the first node should be a population block, the algorithm should contain at least one node of operator block, all nodes should have predecessors and successors, all nodes should be reachable from any other, and all cycles should contain at least one node of population block.
- Step (2) Save or load algorithms or blocks; generate source code of the algorithm; change the display style; automatically arrange the blocks; check the validity of the algorithm; select an algorithm template. After the algorithm is trained, users can generate source code of the algorithm and use it in other modules.
- Step (3) Select multiple labels to determine the type of problems (see *Labels of Algorithms, Problems, and Metrics* for details).
- Step (4) Set the parameters of the problem. Different problems may have different parameters, the details of which can be obtained by hovering over each parameter.
- Step (5) Train the parameters of all blocks of the algorithm on the selected problem. This process may be time-consuming, which may take several days for large

number of blocks, number of variables, population size, and number of function evaluations.

- Step (6) Assess the performance of the trained algorithm on the selected problem.

E. Labels of Algorithms, Problems, and Metrics

Each algorithm, benchmark problem, and metric should be tagged with labels by the comment in the second line of its main function. For example, in the code of `PSO.m`:

```
classdef PSO < ALGORITHM
% <1995> <single> <real/integer> <large/none> <constrained/none>
```

which indicates the types of problems the algorithm can solve. All the labels are

Label	Description
<single>	Single-objective optimization: The problem has a single objective
<multi>	Multi-objective optimization: The problem has two or three objectives
<many>	Many-objective optimization: The problem has four or more objectives
<real>	Continuous optimization: The decision variables are real numbers
<integer>	Integer optimization: The decision variables are integers
<label>	Label optimization: The decision variables are labels
<binary>	Binary optimization: The decision variables are binary numbers
<permutation>	Permutation optimization: All decision variables constitute a permutation
<large>	Large-scale optimization: The problem has 100 or more variables
<constrained>	Constrained optimization: The problem has at least one constraint
<expensive>	Expensive optimization: The objectives are computationally expensive, only a limited number of function evaluations are available
<multimodal>	Multimodal optimization: There exist multiple optimal solutions with similar objective values but considerably different decision vectors, all of which should be found
<sparse>	Sparse optimization: Most variables of the optimal solutions are zero
<dynamic>	Dynamic optimization: The objectives and constraints vary over time
<multitask>	Multitasking optimization: Optimize multiple problems simultaneously, each problem may have multiple objectives and constraints
<bilevel>	Bilevel optimization: Find the feasible and optimal solution for the upper-level problem, where a solution is feasible for the upper-level problem if and only if it is optimal for the lower-level problem
<robust>	Robust optimization: The objectives and constraints are affected by noise, the robust and optimal solutions should be found
<none>	Empty label
<min>	(for metrics only) The metric value is the smaller the better
<max>	(for metrics only) The metric value is the larger the better

An algorithm may have multiple sets of labels, where the Cartesian product between all the label sets constitutes all the types of problems that can be solved by the algorithm. If the label sets of an algorithm are `<single> <real> <constrained/none>`, it will be able to solve single-objective continuous optimization problems with or without constraints. On the other hand, the label sets `<single> <real>` mean that the algorithm can only solve unconstrained problems, the label sets `<single> <real> <constrained>` mean that the algorithm can only solve constrained problems, and the label sets `<single> <real/binary>` mean that the algorithm can solve problems with either real variables or binary variables.

Each algorithm, benchmark problem, and metric should be tagged with at least one label, otherwise it will not be appeared in the lists in the GUI. After selecting multiple labels in the GUI, only the algorithms, benchmark problems, and metrics containing the same labels will be appeared. Details of the label based filter strategy can be found *here*. The labels of all the algorithms and benchmark problems in PlatEMO are referred to *List of Algorithms* and *List of Problems*, respectively.

In addition, each algorithm and benchmark problem can be tagged with a year label like `<2024>`, which enables them to be filtered by year in the lists in the GUI.

IV. Extending PlatEMO

A. ALGORITHM Class

An algorithm should be written as a subclass of `ALGORITHM` and put in the folder `PlatEMO\Algorithms`, which contains the following properties and methods:

Property	Specified by	Description
`parameter`	Users	Parameters of the algorithm
`save`	Users	Number of populations saved in an execution
`run`	Users	Current execution number
`metName`	Users	Names of metrics to calculate
`outputFcn`	Users	Function called in `NotTerminated()`
`pro`	`Solve()`	Problem solved in current execution
`result`	`NotTerminated()`	Populations saved in current execution
`metric`	`NotTerminated()`	Metric values of saved populations
`starttime`	`NotTerminated()`	Used for runtime recording
Method	**Be redefined**	**Description**
`ALGORITHM`	Cannot	Set the properties specified by users Input: Parameter settings like `'Name',Value` Output: `ALGORITHM` object
`Solve`	Cannot	Solve a problem via the algorithm Input: `PROBLEM` object Output: None
`main`	Must	Main procedure of the algorithm Input: `PROBLEM` object Output: None
`NotTerminated`	Cannot	Function called before each iteration in `main()` Input: An array of `SOLUTION` objects, i.e., a population Output: Whether the algorithm terminates (logical)
`ParameterSet`	Cannot	Set the parameter values according to `parameter` Input: Default parameter settings Output: User-specified parameter settings

Each algorithm should inherit the `ALGORITHM` class and redefine the method `main()`. For example, the code of `GA.m` is

```
1 classdef GA < ALGORITHM
2 % <1992><single><real/integer/label/binary/permutation><large/none><constrained/none>
3 % Genetic algorithm
```

```
4  % proC ---  1 --- Probability of crossover
5  % disC --- 20 --- Distribution index of crossover
6  % proM ---  1 --- Expectation of the number of mutated variables
7  % disM --- 20 --- Distribution index of mutation
8
9  %------------------------ Reference ----------------------
10 % J. H. Holland, Adaptation in Natural and Artificial
11 % Systems, MIT Press, 1992.
12 %---------------------------------------------------------
13
14     methods
15         function main(Alg,Pro)
16             [proC,disC,proM,disM] = Alg.ParameterSet(1,20,1,20);
17             P = Pro.Initialization();
18             while Alg.NotTerminated(P)
19                 Q = TournamentSelection(2,Pro.N,FitnessSingle(P));
20                 O = OperatorGA(P(Q),{proC,disC,proM,disM});
21                 P = [P,O];
22                 [~,rank] = sort(FitnessSingle(P));
23                 P = P(rank(1:Pro.N));
24             end
25         end
26     end
27 end
```

The functions of each line are as follows:

Line 1: Inheriting the `ALGORITHM` class;

Line 2: Tagging the algorithm with labels (see *Labels of Algorithms, Problems, and Metrics* for details);

Line 3: Full name of the algorithm;

Lines 4-7: Parameter name --- default value --- description, which are shown in the parameter setting list in the GUI;

Lines 9-12: Reference of the algorithm;

Line 15: Redefining the method of main procedure;

Line 16: Obtaining the parameter values specified by users, where `1,20,1,20` are default values of the four parameters `proC,disC,proM,disM`;

Line 17: Obtaining an initial population by calling a method of the problem;

Line 18: Storing the population and checking whether the algorithm terminates; if so, the algorithm will immediately terminate by throwing an error;

Line 19: Binary tournament based mating selection achieved by a public function;

Line 20: Offspring generation achieved by a public function;

Line 21: Combing the current population with the offspring population;

Line 22: Sorting the solutions based on their fitness calculated by a public function;
Line 23: Retaining half the solutions with better fitness for the next iteration.

In the above codes, the functions `ParameterSet()` and `NotTerminated()` are provided by the `ALGORITHM` class, and the function `Initialization()` is provided by the `PROBLEM` class. Besides, the functions `TournamentSelection()`, `FitnessSingle()`, and `OperatorGA()` are public functions in the folder `PlatEMO\Algorithms\Utility functions`. The following table lists the functions that can be used in algorithms, where the details of them are referred to the comments in their codes. Besides, their techniques for efficiency improvement can be found *here*.

Function Name	Description
`ALGORITHM.NotTerminated`	Function called before each iteration of the algorithm, which stores the current population and check whether the algorithm terminates
`ALGORITHM.ParameterSet`	Set the parameter values specified by users
`PROBLEM.Initialization`	Initialize a population for the problem
`PROBLEM.Evaluation`	Evaluate a population and generate an array of `SOLUTION` object
`CrowdingDistance`	Crowding distance calculation for multi-objective optimization
`FitnessSingle`	Fitness calculation for single-objective optimization
`NDSort`	Non-dominated sorting for multi-objective optimization
`OperatorDE`	The variation operator of differential evolution
`OperatorFEP`	The variation operator of fast evolutionary programming
`OperatorGA`	The variation operators of genetic algorithm
`OperatorGAhalf`	The variation operators of genetic algorithm, where only the first half of offspring solutions are returned
`OperatorPSO`	The variation operator of particle swarm optimization
`RouletteWheelSelection`	Roulette-wheel selection
`TournamentSelection`	Tournament selection
`UniformPoint`	Generate a set of uniformly distributed points

B. PROBLEM Class

A problem should be written as a subclass of `PROBLEM` and put in the folder `PlatEMO\Problems`, which contains the following properties and methods:

Property	Specified by	Description
N	Users	Population size of algorithms
M	Users and Setting()	Number of objectives of the problem
D	Users and Setting()	Number of decision variables of the problem
maxFE	Users	Maximum number of function evaluations
FE	Evaluation()	Number of function evaluations consumed in current execution
maxRuntime	Users	Maximum runtime
encoding	Setting()	Encoding scheme of each variable
lower	Setting()	Lower bound of each variable
upper	Setting()	Upper bound of each variable
optimum	GetOptimum()	Optimal values of the problem, such as the minimum objective value of single-objective optimization problems and a set of points on the Pareto front of multi-objective optimization problems
PF	GetPF()	Pareto front of the problem, such as a 1-D curve of bi-objective optimization problems, a 2-D surface of tri-objective optimization problems, and feasible regions of constrained optimization problems
parameter	Users	Parameters of the problem
Method	**Be redefined**	**Description**
PROBLEM	Cannot	Set the properties specified by users Input: Parameter settings like 'Name',Value Output: ALGORITHM object
Setting	Must	Default settings of the problem Input: None Output: None
Initialization	Can	Initialize a population Input: Population size Output: An array of SOLUTION objects, i.e., a population
Evaluation	Can	Evaluate a population and generate solution objects Input: A matrix consisting of decision vectors Output: An array of SOLUTION objects, i.e., a population
CalDec	Can	Repair invalid solutions in a population Input: A matrix consisting of decision vectors Output: A matrix consisting of repaired decision vectors
CalObj	Must	Calculate the objective values of solutions in a population. All objectives are to be minimized Input: A matrix consisting of decision vectors Output: A matrix consisting of objective values
CalCon	Can	Calculate the constraint violations of solutions in a

		population. A constraint is satisfied if and only if the constraint violation is not positive Input: A matrix consisting of decision vectors Output: A matrix consisting of constraint violations
`CalGrad`	Can	Calculate the gradients of a solution on objectives and constraints Input: A decision vector Output 1: Jacobian matrix of objectives Output 2: Jacobian matrix of constraints
`GetOptimum`	Can	Generate the optimal values and store in `optimum` Input: The number of optimal values Output: Optimal values (a matrix)
`GetPF`	Can	Generate the Pareto front and store in `PF` Input: None Output: Data for plotting the Pareto front (a matrix or cell)
`CalMetric`	Can	Calculate the metric value of a population Input 1: Metric name Input 2: An array of `SOLUTION` objects, i.e., a population Output: Metric value (scalar)
`DrawDec`	Can	Display the decision variables of a population Input: An array of `SOLUTION` objects, i.e., a population Output: None
`DrawObj`	Can	Display the objective values of a population Input: An array of `SOLUTION` objects, i.e., a population Output: None
`ParameterSet`	Cannot	Set the parameter values according to `parameter` Input: Default parameter settings Output: User-specified parameter settings

Each benchmark problem should inherit the `PROBLEM` class and redefine the methods `Setting()` and `CalObj()`. For example, the code of `SOP_F1.m` is

```
classdef SOP_F1 < PROBLEM
% <1999><single><real><expensive/none>
% Sphere function

%------------------------ Reference ------------------------
% X. Yao, Y. Liu, and G. Lin, Evolutionary programming made
% faster, IEEE Transactions on Evolutionary Computation,
% 1999, 3(2): 82-102.
%-------------------------------------------------------------

    methods
```

```
12          function Setting(obj)
13              obj.M = 1;
14              if isempty(obj.D); obj.D = 30; end
15              obj.lower = zeros(1,obj.D) - 100;
16              obj.upper = zeros(1,obj.D) + 100;
17              obj.encoding = ones(1,obj.D);
18          end
19          function PopObj = CalObj(obj,PopDec)
20              PopObj = sum(PopDec.^2,2);
21          end
22      end
23  end
```

The functions of each line are as follows:

Line 1: Inheriting the `PROBLEM` class;

Line 2: Tagging the problem with labels (see *Labels of Algorithms, Problems, and Metrics* for details);

Line 3: Full name of the problem;

Lines 5-9: Reference of the problem;

Line 12: Redefining the method of default parameter settings;

Line 13: Setting the number of objectives;

Line 14: Setting the number of decision variables if it is not specified by users;

Lines 15-16: Setting the lower bounds and upper bounds of decision variables;

Line 17: Setting the encoding schemes of decision variables;

Line 19: Redefining the method of calculating objective values;

Line 20: Calculating the objective values of solutions in a population.

The default method `Initialization()` randomly initializes a population. This method can be redefined to specify a novel initialization strategy. For example, `Sparse_NN.m` initializes a population in which half the decision variables are zero:

```
function Population = Initialization(obj,N)
    if nargin < 2; N = obj.N; end
    PopDec = (rand(N,obj.D)-0.5)*2.*randi([0 1],N,obj.D);
    Population = SOLUTION(PopDec);
end
```

The default method `CalDec()` repairs invalid solutions in a population, where each decision variable will be set to the boundary values if it is larger than the upper bound or smaller than the lower bound. This method can be redefined to specify a novel repair strategy. For example, `MOKP.m` repairs solutions that exceed the capacity, so that no constraint needs to be defined in this problem:

```
function PopDec = CalDec(obj,PopDec)
    C = sum(obj.W,2)/2;
    [~,rank] = sort(max(obj.P./obj.W));
    for i = 1 : size(PopDec,1)
        while any(obj.W*PopDec(i,:)'>C)
            k = find(PopDec(i,rank),1);
            PopDec(i,rank(k)) = 0;
        end
    end
end
```

The default method `CalCon()` returns zero as the constraint violation of the solutions in a population, i.e., all the solutions are feasible. This method can be redefined to specify constraint functions for the problem. For example, `CF4.m` calculates a constraint for each solution:

```
function PopCon = CalCon(obj,X)
    t = X(:,2)-sin(6*pi*X(:,1)+2*pi/size(X,2))-0.5*X(:,1)+0.25;
    PopCon = -t./(1+exp(4*abs(t)));
end
```

Use `all(PopCon<=0,2)` to determine whether each solution is feasible or not. Note that equality constraints should be converted into inequality constraints, the details of which can be found in Section 3.2 of *this paper*. The default method `Evaluation()` calls `CalDec()`, `CalObj()`, and `CalCon()` in sequence to instantiate `SOLUTION` objects, and also adds the number of consumed function evaluations `FE`. This method can be redefined to perform solution repair, objective calculation, and constraint calculation in a single function, where `CalDec()`, `CalObj()`, and `CalCon()` will not be called anymore. For example, `MW2.m` calculates objective values and constraint violations in a single function:

```
function Population = Evaluation(obj,varargin)
    X = varargin{1};
    X=max(min(X,repmat(obj.upper,size(X,1),1)),repmat(obj.lower,size(X,1),1));
    z=1-exp(-10*(X(:,obj.M:end)-(repmat(obj.M:obj.D,size(X,1),1)-1)/obj.D).^2);
    g = 1+sum((1.5+(0.1/obj.D)*z.^2-1.5*cos(2*pi*z)),2);
    PopObj(:,1) = X(:,1);
    PopObj(:,2) = g.*(1-PopObj(:,1)./g);
    L = sqrt(2)*PopObj(:,2)-sqrt(2)*PopObj(:,1);
    PopCon = sum(PopObj,2)-1-0.5*sin(3*pi*l).^8;
    Population = SOLUTION(X,PopObj,PopCon,varargin{2:end});
    obj.FE = obj.FE+length(Population);
end
```

The default method `CalGrad()` estimates the gradients of objectives and constraints via finite difference, while this method can be redefined to calculate gradients more accurately. The method `GetOptimum()` can be redefined to specify the optimal values of the problem, which are used for metric calculation. For example, `SOP_F8.m` returns the optimal value of the objective function:

```
function R = GetOptimum(obj,N)
    R = -418.9829*obj.D;
end
```

and `DTLZ2.m` returns a set of uniformly distributed points on the Pareto front:

```
function R = GetOptimum(obj,N)
    R = UniformPoint(N,obj.M);
    R = R./repmat(sqrt(sum(R.^2,2)),1,obj.M);
end
```

The strategies for sampling points on different Pareto fronts can be found *here*. The method `GetPF()` can be redefined to specify the Pareto front or feasible regions of multi-objective optimization problems for the visualization achieved in `DrawObj()`. For example, `DTLZ2.m` returns the data for plotting the 2-D and 3-D Pareto fronts:

```
function R = GetPF(obj)
    if obj.M == 2
        R = obj.GetOptimum(100);
    elseif obj.M == 3
        a = linspace(0,pi/2,10)';
        R = {sin(a)*cos(a'),sin(a)*sin(a'),cos(a)*ones(size(a'))};
    else
        R = [];
    end
end
```

and `MW1.m` returns the data for plotting the feasible regions:

```
function R = GetPF(obj)
    [x,y]= meshgrid(linspace(0,1,400),linspace(0,1.5,400));
    z    = nan(size(x));
    fes  = x+y-1-0.5*sin(2*pi*(sqrt(2)*y-sqrt(2)*x)).^8 <= 0;
    z(fes&0.85*x+y>=1) = 0;
    R    = {x,y,z};
end
```

The default method `CalMetric()` passes a population and the optimal values `optimum` to a metric function to calculate the metric value. This method can be

redefined to pass different variables to metric functions. For example, `SMMOP1.m` passes the Pareto optimal set rather than the points on the Pareto front when calculating the metric value of IGDX:

```
function score = CalMetric(obj,metName,Population)
    switch metName
        case 'IGDX'
            score = feval(metName,Population,obj.POS);
        otherwise
            score = feval(metName,Population,obj.optimum);
    end
end
```

The default method `DrawDec()` displays the decision variables of a population, which is used for the visualization of results in the GUI. This method can be redefined to specify a novel visualization method. For example, `TSP.m` displays the route of the best solution:

```
function DrawDec(obj,P)
    [~,best] = min(P.objs);
    Draw(obj.R(P(best).dec([1:end,1]),:),'-k','LineWidth',1.5);
    Draw(obj.R);
end
```

The default method `DrawObj()` displays the objective values of a population, which is used for the visualization of results in the GUI. This method can be redefined to specify a novel visualization method. For example, `Sparse_CD.m` adds labels to the axes:

```
function DrawObj(obj,P)
    Draw(P.objs,{'Kernel k-means','Ratio cut',[]});
end
```

where `Draw()` is a function in the folder `PlatEMO\GUI` for displaying data.

C. SOLUTION Class

A `SOLUTION` object denotes an individual, and an array of `SOLUTION` objects denote a population. The `SOLUTION` class contains the following properties and methods:

Property	Specified by	Description
`dec`	Users	Decision variables of the solution
`obj`	`SOLUTION()`	Objective values of the solution
`con`	`SOLUTION()`	Constraint violations of the solution

add	adds()	Additional properties (e.g., velocity) of the solution

Method	Description
SOLUTION	Generate SOLUTION objects Input 1: A matrix consisting of decision vectors Input 2: A matrix consisting of objective values Input 3: A matrix consisting of constraint violations Input 4: A matrix consisting of additional properties Output: An array of SOLUTION objects
decs	Get the decision variables of multiple solutions Input: None Output: A matrix consisting of decision vectors
objs	Get the objective values of multiple solutions Input: None Output: A matrix consisting of objective values
cons	Get the constraint violations of multiple solutions Input: None Output: A matrix consisting of constraint violations
adds	Set and get the additional properties of multiple solutions Input: Default additional properties Output: A matrix consisting of additional properties
best	Get the feasible and best solution for single-objective optimization, or the feasible and non-dominated solutions for multi-objective optimization Input: None Output: A subarray of best SOLUTION objects in the population

For example, the following code generates a population with ten solutions, then gets the objective matrix of the best solutions in the population:

```
Population = SOLUTION(rand(10,5),rand(10,1),zeros(10,1));
BestObjs   = Population.best.objs
```

Note that SOLUTION() should be called only in the method Evaluation() of PROBLEM class.

D. Whole Procedure of One Run

The following code uses the genetic algorithm to solve the sphere function:

```
Alg = GA();
Pro = SOP_F1();
Alg.Solve(Pro);
```

where the functions called in the execution of Alg.Solve(Pro) are as follows.



E. Metric Function

A metric should be written as a function and put in the folder `PlatEMO\Metrics`. For example, the code of `IGD.m` is

```
function score = IGD(Population,optimum)
% <min> <multi/many> <real/integer/label/binary/permutation> <large/none> <constrained/none> <expensive/none> <multimodal/none> <sparse/none> <dynamic/none> <robust/none>
% Inverted generational distance

%------------------------ Reference ----------------------
% C. A. Coello Coello and N. C. Cortes, Solving
% multiobjective optimization problem using an artificial
% immune system, Genetic Programming and Evolvable
```

```
% Machines, 2005, 6(2): 163-190.
%--------------------------------------------------------------

    PopObj = Population.best.objs;
    if size(PopObj,2) ~= size(optimum,2)
        score = nan;
    else
        score = mean(min(pdist2(optimum,PopObj),[],2));
    end
end
```

The functions of each line are as follows:

Line 1: Function declaration, where the first input is a population (i.e., an array of `SOLUTION` objects), the second input is the optimums of a problem (i.e., the `optimum` property of the problem), and the output is the metric value;

Line 2: Tagging the metric with labels (see *Labels of Algorithms, Problems, and Metrics* for details); note that `<min>` or `<max>` should be the first label;

Line 3: Full name of the metric;

Lines 5-10: Reference of the metric;

Line 12: Obtaining the feasible and non-dominated solutions in the population;

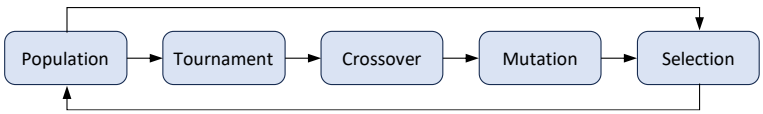
Lines 13-14: Returns `nan` if there is no feasible solution in the population;

Lines 15-16: Returns the IGD value of the feasible and non-dominated solutions.

F. Creating NeuroEAs

NeuroEAs provide a flexible framework to create new algorithms via a directed and weighted cyclic graph. Each node in the graph is a population processing block like crossover, mutation, and selection, and each edge in the graph determines the transmission directions and ratios of solutions between nodes. Each node has many parameters that can be automatically trained, and a well-trained NeuroEA can achieve outstanding performance on training problems.

A NeuroEA is defined by an array of `BLOCK` objects and adjacency matrix. For example, a simple NeuroEA represented by

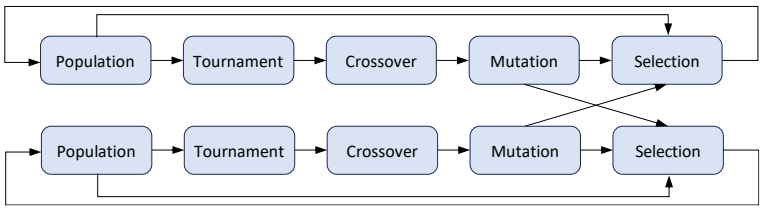


can be defined and executed by

```
addpath('Algorithms\NeuroEA');
```

```
Blocks = [Block_Population
          Block_Tournament(200,10)
          Block_Crossover(2,5)
          Block_Mutation(5)
          Block_Selection(100)];
Graph = [0 1 0 0 1
         0 0 1 0 0
         0 0 0 1 0
         0 0 0 0 1
         1 0 0 0 0];
platemo('algorithm',{@NeuroEA,Blocks,Graph},'problem',@SOP_F1);
```

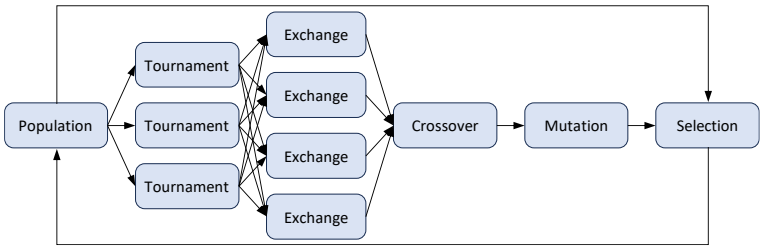
A multi-population NeuroEA represented by



can be defined and executed by

```
addpath('Algorithms\NeuroEA');
Blocks = [Block_Population
          Block_Tournament(100,10)
          Block_Crossover(2,5)
          Block_Mutation(5)
          Block_Selection(100)
          Block_Population
          Block_Tournament(100,10)
          Block_Crossover(2,5)
          Block_Mutation(5)
          Block_Selection(100)];
Graph = [0 1 0 0 1 0 0 0 0 0
         0 0 1 0 0 0 0 0 0 0
         0 0 0 1 0 0 0 0 0 0
         0 0 0 0 1 0 0 0 0 1
         1 0 0 0 0 0 0 0 0 0
         0 0 0 0 0 0 1 0 0 1
         0 0 0 0 0 0 0 1 0 0
         0 0 0 0 0 0 0 0 1 0
         0 0 0 0 0 0 0 0 0 1
         0 0 0 0 0 1 0 0 0 0];
platemo('algorithm',{@NeuroEA,Blocks,Graph},'problem',@SOP_F1);
```

A complex NeuroEA represented by



can be defined and executed by

```
addpath('Algorithms\NeuroEA');
Blocks = [Block_Population
          Block_Tournament(200,10)
          Block_Tournament(200,10)
          Block_Tournament(200,10)
          Block_Exchange(3)
          Block_Exchange(3)
          Block_Exchange(3)
          Block_Exchange(3)
          Block_Crossover(2,5)
          Block_Mutation(5)
          Block_Selection(100)];
Graph = [0 1 1 1 0 0 0 0 0 0 1
         0 0 0 0 1/4 1/4 1/4 1/4 0 0 0
         0 0 0 0 1/4 1/4 1/4 1/4 0 0 0
         0 0 0 0 1/4 1/4 1/4 1/4 0 0 0
         0 0 0 0 0 0 0 0 1 0 0
         0 0 0 0 0 0 0 0 1 0 0
         0 0 0 0 0 0 0 0 1 0 0
         0 0 0 0 0 0 0 0 1 0 0
         0 0 0 0 0 0 0 0 0 1 0
         0 0 0 0 0 0 0 0 0 0 1
         1 0 0 0 0 0 0 0 0 0 0];
platemo('algorithm',{@NeuroEA,Blocks,Graph},'problem',@SOP_F1);
```

Each block in NeuroEAs is an instance of a block class. A block class should be written as a subclass of `BLOCK` and put in the folder `PlatEMO\Algorithms\NeuroEA`, which contains the following properties and methods:

Property	Specified by	Description
`parameter`	`ParameterSet()`	Parameters of the block
`lower`	Constructor	Lower bound of each parameter
`upper`	Constructor	Upper bound of each parameter
`output`	`Main()`	Current output population of the block

nextOut	Gather()	Index of next output solution
trainTime	Users	Number of training times
Method	**Be redefined**	**Description**
Constructor	Must	Set the properties specified by users Input: Settings of hyperparameters Output: BLOCK object
Main	Must	Main procedure of the block Input 1: PROBLEM object Input 2: Precursor blocks of this block Input 3: Ratio of solutions gathered from each precursor block Output: None
ParameterAssign	Can	Assign parameters to properties Input: None Output: None
ParameterSet	Cannot	Set the parameters of multiple blocks Input: A vector of parameters for multiple blocks Output: None
parameters	Cannot	Get the parameters of multiple blocks Input: None Output: A vector of parameters for multiple blocks
lowers	Cannot	Get the lower bounds of parameters of multiple blocks Input: None Output: A vector of lower bounds
uppers	Cannot	Get the upper bounds of parameters of multiple blocks Input: None Output: A vector of upper bounds
Gather	Cannot	Gather the output from precursor blocks Input 1: PROBLEM object Input 2: Precursor blocks of this block Input 3: Ratio of solutions gathered from each precursor block Input 4: Gathering SOLUTION objects or decision variables Input 5: A value *k*, where the number of gathered solutions should be a multiple of *k* Output: SOLUTION objects or decision matrix
Validity	Cannot	Check the validity of a NeuroEA Input: Adjacency matrix Output: None

Each block class should inherit the `BLOCK` class and redefine the constructor and the method `Main()`. For example, the code of `Block_Mutation.m` is

```
1  classdef Block_Mutation < BLOCK
2  % Unified mutation for real variables
3  % nSets --- 5 --- Number of parameter sets
4
5      properties
6          nSets;
7          Weight;
8          Fit;
9          nDec = 1;
10     end
11     methods
12         function obj = Block_Mutation(nSets)
13             obj.nSets = nSets;
14             obj.lower = repmat([0 1e-20],1,nSets);
15             obj.upper = repmat([1 5],1,nSets);
16             obj.parameter = unifrnd(obj.lower,ones(1,2*nSets));
17             obj.ParameterAssign();
18         end
19         function ParameterAssign(obj)
20             obj.Weight = reshape(obj.parameter,[],obj.nSets)';
21             obj.Weight(:,end) = obj.Weight(:,end)./obj.nDec;
22             obj.Weight = [obj.Weight;0,max(0,1-sum(obj.Weight(:,end)))];
23             obj.Fit = cumsum(obj.Weight(:,end));
24             obj.Fit = obj.Fit./max(obj.Fit);
25         end
26         function Main(obj,Problem,Precursors,Ratio)
27             ParentDec = obj.Gather(Problem,Precursors,Ratio,2,1);
28             if size(ParentDec,2) ~= obj.nDec
29                 obj.nDec = size(ParentDec,2);
30                 obj.ParameterAssign();
31             end
32             r = ParaSampling(size(ParentDec),obj.Weight(:,1),obj.Fit);
33             obj.output = ParentDec + repmat(Problem.upper-...
                           Problem.lower,size(ParentDec,1),1).*r;
34         end
35     end
36 end
```

The functions of each line are as follows:

Line 1: Inheriting the `BLOCK` class;

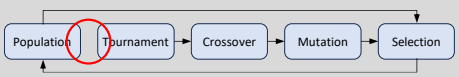
Line 2: Full name of the block;

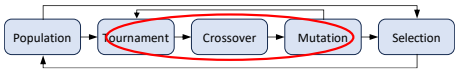
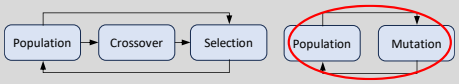
Line 3: Hyperparameter name --- default value --- description, which are shown in the block setting panel in the GUI;
Lines 5-10: Special properties of the block;
Line 12: Redefining the constructor;
Line 13: Setting a special property;
Lines 14-15: Setting the lower bounds and upper bounds of parameters;
Line 16: Randomly generating parameters;
Line 17: Calling the method `ParameterAssign()` to assign parameters to special properties;
Lines 19-25: Redefining the method of assigning parameters to special properties; this method is automatically called by the method `ParameterSet()`;
Line 26: Redefining the method of main procedure;
Line 27: Gathering solutions (i.e., a decision matrix) from precursor blocks;
Lines 28-33: Do mutation to generate new solutions (i.e., a decision matrix); this method is automatically called by `NeuroEA.m`.

Currently, seven types of blocks (i.e., subclasses of `BLOCK`) are provided for creating NeuroEAs, including:

Block	Description
`Block_Population`	Storing solutions without any processing
`Block_Crossover`	Mating multiple solutions to generate a new solution
`Block_Mutation`	Mutating one solution
`Block_Exchange`	Exchanging multiple solutions to generate a new solution
`Block_Kopt`	Flipping part of a solution for permutation optimization
`Block_Tournament`	Tournament selection for mating selection
`Block_Selection`	Retaining part of solutions for environmental selection

The details of these blocks and their hyperparameters can be found in *this paper*. These blocks can be arbitrarily connected to form a graph representing a NeuroEA, but should not have the following invalid scenarios:

Invalid scenario	Example
The first block is not a population	
The graph has no variation operator	
A block has no predecessor block	
A block has no successor block	

A block has self-loop	
A cycle has no population block	
Blocks are isolated	

The validity of a NeuroEA can be checked by calling the method `Validity()`. For example, the following code checks the validity of a NeuroEA defined by `Blocks` and `Graph`, and outputs the invalid blocks obtained from `err` thrown by `Validity()`:

```
try
    Block.Validity(Graph);
catch err
    switch err.identifier
        case 'BLOCK:NoPopulation'

        case 'BLOCK:NoOperator'

        case 'BLOCK:NoInput'
            str2num(err.cause{1}.message)
        case 'BLOCK:NoOutput'
            str2num(err.cause{1}.message)
        case 'BLOCK:SelfLoop'
            str2num(err.cause{1}.message)
        case 'BLOCK:InfLoop'
            str2num(err.cause{1}.message)
        case 'BLOCK:Isolation'
            str2num(err.cause{1}.message)
    end
end
```

The input of the constructor of the `BLOCK` class contains the hyperparameters of a block, which determines some special properties of the block. In addition, a block has many parameters stored in the property `parameter` that also determines some special properties, which can be trained to achieve outstanding performance. The training procedure can be achieved using the *Creation Module* or the following codes:

```
addpath('Algorithms\NeuroEA');
Blocks = [Block_Population
          Block_Tournament(200,10)
          Block_Crossover(2,5)
          Block_Mutation(5)
          Block_Selection(100)];
```

```
Graph = [0 1 0 0 1
         0 0 1 0 0
         0 0 0 1 0
         0 0 0 0 1
         1 0 0 0 0];
function y = Fcn(x,data)
    data{1}.ParameterSet(x);
    for i = 1 : 3
        [~,obj] = platemo('algorithm',...
                          {@NeuroEA,data{1},data{2}},...
                          'problem',@SOP_F1,...
                          'outputFcn',@(~,~)[]);
        s(i) = min(obj);
    end
    y = mean(s);
end
platemo('algorithm',@GA,'objFcn',@Fcn,...
        'lower',Blocks.lowers,...
        'upper',Blocks.uppers,...
        'D',length(Blocks.lowers),...
        'data',{Blocks,Graph},'N',30,'save',1);
```

That is, training the NeuroEA is regarded as an optimization problem solved by `@GA`, where the variables are the parameters of all the blocks and the objective function `Fcn()` calculates the performance of the NeuroEA on `@SOP_F1` averaged over three runs.

V. List of Algorithms

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
1	ABC	Artificial bee colony algorithm	√			√	√				√	√							
2	AB-SAEA	Adaptive Bayesian based surrogate-assisted evolutionary algorithm		√	√	√	√						√						
3	AC-MMEA	Adaptive merging and coordinated offspring generation based multi-modal multi-objective evolutionary algorithm		√		√	√				√			√	√				
4	ACO	Ant colony optimization	√							√	√								
5	Adam	Adaptive moment estimation	√			√					√								
6	AdaW	Evolutionary algorithm with adaptive weights		√	√	√	√	√	√	√									
7	ADSAPSO	Adaptive dropout based surrogate-assisted particle swarm optimization		√	√	√	√						√						
8	AE-NSGA-II	Autoencoding NSGA-II		√		√	√	√	√	√		√				√			
9	AESSPSO	Adaptive exploration state-space particle swarm optimization	√			√	√				√	√							
10	AFSEA	Adjoint feature-selection-based evolutionary algorithm		√		√	√		√		√	√			√				
11	AGE-II	Approximation-guided evolutionary multi-objective algorithm II		√		√	√	√	√	√									
12	AGE-MOEA	Adaptive geometry estimation-based many-objective evolutionary algorithm		√	√	√	√	√	√	√		√							
13	AGE-MOEA-II	Adaptive geometry estimation-based many-objective evolutionary algorithm II		√	√	√	√	√	√	√		√							
14	AGSEA	Automated guiding vector selection-based evolutionary algorithm		√		√	√		√		√	√			√				
15	AMG-PSL	Adaptive multi-granular Pareto-optimal subspace learning		√		√	√		√		√	√			√				
16	A-NSGA-III	Adaptive NSGA-III		√	√	√	√	√	√	√		√							
17	APSEA	Adaptive population sizing based evolutionary algorithm		√		√	√	√	√	√		√							
18	AR-MOEA	Adaptive reference points based multi-objective evolutionary algorithm		√	√	√	√	√	√	√		√							
19	AutoV	Automated design of variation operators	√	√		√	√				√	√							
20	AVG-SAEA	Adaptive variable grouping based surrogate-assisted evolutionary algorithm		√		√	√				√		√						
21	BCE-IBEA	Bi-criterion evolution based IBEA		√	√	√	√	√	√	√									
22	BCE-MOEA/D	Bi-criterion evolution based MOEA/D		√	√	√	√	√	√	√									
23	BFGS	A quasi-Newton method proposed by Broyden, Fletcher, Goldfarb, and Shanno	√			√					√								
24	BiCo	Bidirectional coevolution constrained multiobjective evolutionary algorithm		√		√	√	√	√	√		√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
25	BiGE	Bi-goal evolution			√	√	√	√	√	√									
26	BLEAQII	Bilevel evolutionary algorithm based on quadratic approximations II		√		√						√						√	
27	BL-SAEA	Bi-level surrogate modelling based evolutionary algorithm		√		√						√						√	
28	BSPGA	Binary space partition tree based genetic algorithm	√						√		√	√							
29	C3M	Constraint, multiobjective, multi-stage, multi-constraint evolutionary algorithm		√		√	√	√	√	√		√							
30	CAEAD	Dual-population evolutionary algorithm based on alternative evolution and degeneration		√		√	√	√	√	√		√							
31	CA-MOEA	Clustering based adaptive multi-objective evolutionary algorithm		√		√	√	√	√	√									
32	CCGDE3	Cooperative coevolution GDE3		√		√	√				√								
33	CCMO	Coevolutionary constrained multi-objective optimization framework		√		√	√	√	√	√		√							
34	c-DPEA	Constrained dual-population evolutionary algorithm		√		√	√	√	√	√		√							
35	CGLP	Correlation-guided layered prediction		√		√	√	√	√	√						√			
36	CI-EMO	Composite indicator-guided infilling sampling for expensive multi-objective optimization		√	√	√							√						
37	CKAnD	Constrained Kriging-assisted AnD		√	√	√						√	√						
38	CLIA	Evolutionary algorithm with cascade clustering and reference point incremental learning		√	√	√	√	√	√	√									
39	CMaDPPs	Constrained many-objective optimization with determinantal point processes		√	√	√	√	√	√	√		√							
40	CMA-ES	Covariance matrix adaptation evolution strategy	√			√	√				√	√							
41	CMDEIPCM	Constrained multiobjective differential evolution algorithm with an infeasible proportion control mechanism		√		√	√				√	√							
42	CMEA	Cooperative multi-population evolutionary algorithm		√		√	√				√			√	√				
43	CMEGL	Constrained evolutionary multitasking with global and local auxiliary tasks		√		√	√	√	√	√		√							
44	CMME	Constrained many-objective evolutionary algorithm with enhanced mating and environmental selections		√		√	√	√	√	√		√							
45	CMMO	Coevolutionary multi-modal multi-objective optimization framework		√		√	√	√	√	√				√					
46	CMOBR	Constrained multiobjective optimization via both constraint and objective relaxations		√	√	√	√					√							
47	CMOCEA-DDAP	Constrained multi-objective co-evolutionary algorithm with a dynamic diversity-enhanced auxiliary population		√		√	√					√							
48	CMOCSO	Competitive and cooperative swarm optimization constrained multi-objective optimization algorithm		√		√					√	√							
49	CMODE-FTR	Constrained multiobjective differential evolution based on the fusion of two rankings		√		√	√					√							
50	CMODRL	Constrained multiobjective optimization via deep reinforcement learning		√		√	√	√	√	√		√							
51	CMOEA-2S	Constrained MOEA with two types of evolution stages		√		√						√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
52	CMOEA-ACRT	Constrained MOEA with comprehensive ranking technique		√		√						√							
53	CMOEA-AOP	Automated operator portfolio based constrained MOEA		√		√	√	√	√	√		√							
54	CMOEA-CD	Constraint-Pareto dominance and diversity enhancement strategy based constrained MOEA		√	√	√	√	√	√	√		√							
55	C-MOEA/D	Constraint-MOEA/D		√	√	√	√	√	√	√		√							
56	CMOEA-MS	Constrained multiobjective evolutionary algorithm with multiple stages		√		√	√	√	√	√		√							
57	CMOEA-MSG	Multi-stage constrained multi-objective evolutionary algorithm		√		√	√					√							
58	CMOEA-RDA	Constrained MOEA with role dynamic adjustment		√		√						√							
59	CMOEBOD	Constrained multiobjective evolutionary Bayesian optimization based on decomposition		√	√	√						√	√						
60	CMOEMT	Constrained multi-objective optimization based on evolutionary multitasking optimization		√		√						√							
61	CMOFLCS	Constrained multi-objective optimization with fuzzy logic control system		√		√						√							
62	CMOES	Constrained multi-objective optimization based on even search		√		√	√	√	√	√		√							
63	CMOPSO	Competitive mechanism based multi-objective particle swarm optimizer		√		√	√												
64	CMOQLMT	Constrained multi-objective optimization based on Q-learning and multitasking		√		√						√							
65	CMOSMA	Constrained multi-objective evolutionary algorithm with self-organizing map		√	√	√	√					√							
66	CNSDE/DVC	Constrained nondominated sorting differential evolution based on decision variable classification		√		√	√												√
67	CoMMEA	Coevolutionary multimodal multi-objective evolutionary algorithm		√		√	√	√	√	√				√					
68	CPCMO	Constraint, multiobjective, multi-stage, multi-constraint evolutionary algorithm		√		√	√	√	√	√		√							
69	CPS-MOEA	Classification and Pareto domination based multi-objective evolutionary		√		√	√						√						
70	CSEA	Classification based surrogate-assisted evolutionary algorithm		√	√	√							√						
71	CSEMT	Constraints separation based evolutionary multitasking		√		√	√	√	√	√		√							
72	CSO	Competitive swarm optimizer	√			√	√				√	√							
73	C-TAEA	Two-archive evolutionary algorithm for constrained MOPs		√	√	√	√	√	√	√		√							
74	C-TSEA	Constrained two-stage evolutionary algorithm		√	√	√	√	√	√	√		√							
75	DAEA	Duplication analysis based evolutionary algorithm		√					√										
76	DBEMTO	Double-balanced evolutionary multi-task optimization		√		√	√	√	√	√		√							
77	DCNSGA-III	Dynamic constrained NSGA-III		√	√	√	√	√	√	√		√							
78	DE	Differential evolution	√			√	√				√	√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
79	DEA-GNG	Decomposition based evolutionary algorithm guided by growing neural gas		√	√	√	√	√	√	√									
80	DGEA	Direction guided evolutionary algorithm		√	√	√	√				√								
81	DirHV-EI	Expected direction-based hypervolume improvement		√	√	√	√						√						
82	DISK	Distribution-based Kriging-assisted evolutionary algorithm		√	√	√	√						√						
83	DISKplus	Distribution-based Kriging-assisted constrained evolutionary algorithm		√	√	√	√					√	√						
84	DKCA	Dynamic knowledge-guided coevolutionary algorithm		√		√			√		√	√			√				
85	DM-MOEA	Dual model based multi-objective evolutionary algorithm		√		√	√		√		√	√			√	√			
86	DMOEA-eC	Decomposition-based multi-objective evolutionary algorithm with the e-constraint framework		√		√	√	√	√	√									
87	dMOPSO	MOPSO based on decomposition		√		√	√												
88	DN-NSGA-II	Decision space based niching NSGA-II		√		√	√							√					
89	DNSGA-II	Dynamic NSGA-II		√		√	√	√	√	√						√			
90	DOA	Dandelion optimization algorithm	√			√	√				√	√							
91	DPCPRA	Dual-population with dynamic constraint processing and resource allocating		√		√	√	√	√	√		√							
92	DP-PPS	Tri-population based push and pull search		√		√						√							
93	DPVAPS	Dual-population with variable auxiliary population size		√		√	√				√	√							
94	DRLOS-EMCMO	EMCMO with deep reinforcement learning-assisted operator selection		√		√	√	√	√	√		√							
95	DRL-SAEA	Deep reinforcement learning-based expensive constrained evolutionary algorithm		√		√						√	√						
96	DSOCOL	Dual-swarm optimizer with collaborative orthogonal learning		√		√					√	√							
97	DSPCMDE	Dynamic selection preference-assisted constrained multiobjective differential evolution		√		√	√					√							
98	DSSEA	Dynamic subspace search-based evolutionary algorithm		√	√	√	√				√	√							
99	DVCEA	Decision variables classification-based evolutionary algorithm		√	√	√	√				√	√							
100	DWU	Dominance-weighted uniformity multi-objective evolutionary algorithm		√		√	√	√	√	√									
101	EAG-MOEA/D	External archive guided MOEA/D		√		√	√	√	√	√									
102	ECPO	Electric charged particles optimization	√			√	√				√	√							
103	EDN-ARMOEA	Efficient dropout neural network based AR-MOEA		√	√	√	√						√						
104	EFR-RR	Ensemble fitness ranking with a ranking restriction scheme		√	√	√	√	√	√	√									
105	EGES	Efficient grouping evolutionary search		√	√	√					√		√						
106	EGO	Efficient global optimization	√			√	√						√						
107	EIM-EGO	Expected improvement matrix based efficient global optimization		√		√	√						√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
108	EMCMMS	Evolutionary multitasking with a cooperative multistep mutation strategy		√		√	√	√	√	√		√							
109	EMCMO	Evolutionary multitasking-based constrained multiobjective optimization		√		√	√	√	√	√		√							
110	EMMOEA	Expensive multi-/many-objective evolutionary algorithm		√		√	√						√						
111	e-MOEA	Epsilon multi-objective evolutionary algorithm		√	√	√	√	√	√	√									
112	EMOSKT	Evolutionary multi-objective optimization with sparsity knowledge transfer		√		√			√		√	√			√		√		
113	EM-SAEA	Ensemble-based surrogate model-assisted evolutionary algorithm		√	√	√						√	√						
114	EMyO/C	Evolutionary many-objective optimization algorithm with clustering-based		√	√	√	√												
115	ENS-MOEA/D	Ensemble of different neighborhood sizes based MOEA/D		√	√	√	√												
116	ESBCEO	Bayesian co-evolutionary optimization based entropy search		√		√							√						
117	FDSEA	Frequency domain search-based evolutionary algorithm		√		√	√				√								
118	FDV	Fuzzy decision variable framework with various internal optimizers		√	√	√	√				√								
119	FEP	Fast evolutionary programming	√			√	√				√	√							
120	FLEA	Fast sampling based evolutionary algorithm		√	√	√					√								
121	FRCG	Fletcher-Reeves conjugate gradient	√			√					√								
122	FRCGM	Fletcher-Reeves conjugate gradient (for multi-objective optimization)		√	√	√					√	√							
123	FROFI	Feasibility rule with the incorporation of objective function information	√			√	√				√	√							
124	GA	Genetic algorithm	√			√	√	√	√	√	√	√							
125	GCNMOEA	Graph convolutional network based multi-objective evolutionary algorithm		√		√	√												
126	GDE3	Generalized differential evolution 3		√		√	√					√							
127	GDVTSF	Generational difference vector based tri-entropy structure framework		√		√					√								
128	GFM-MOEA	Generic front modeling based multi-objective evolutionary algorithm		√	√	√	√	√	√	√									
129	GLMO	Grouped and linked mutation operator algorithm		√		√	√				√								
130	g-NSGA-II	g-dominance based NSGA-II		√		√	√	√	√	√									
131	GPSO	Gradient based particle swarm optimization algorithm	√			√					√	√							
132	GPSOM	Gradient based particle swarm optimization algorithm (for multi-objective optimization)		√	√	√					√	√							
133	GrEA	Grid-based evolutionary algorithm			√	√	√	√	√	√									
134	GWASF-GA	Global weighting achievement scalarizing function genetic algorithm		√		√	√	√	√	√									
135	GWO	Grey wolf optimizer	√			√	√				√	√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
136	HEA	Hyper-dominance based evolutionary algorithm		√	√	√			√	√									
137	HeE-MOEA	Multiobjective evolutionary algorithm with heterogeneous ensemble based infill criterion		√		√	√						√						
138	HHC-MMEA	Hybrid hierarchical clustering based multi-modal multi-objective evolutionary algorithm		√		√					√			√	√				
139	hpaEA	Hyperplane assisted evolutionary algorithm		√	√	√	√	√	√	√									
140	HREA	Hierarchy ranking based evolutionary algorithm		√		√	√							√					
141	HypE	Hypervolume estimation algorithm		√	√	√	√	√	√	√									
142	IBEA	Indicator-based evolutionary algorithm		√	√	√	√	√	√	√									
143	ICMA	Indicator based constrained multi-objective algorithm		√		√	√					√							
144	I-DBEA	Improved decomposition-based evolutionary algorithm		√	√	√	√	√	√	√		√							
145	ILCMO	Indicator-based evolutionary algorithm for large-scale constrained multi-objective optimization		√	√	√	√				√	√							
146	IM-C-MOEA/D	Inverse modeling constrained MOEA/D		√		√	√				√	√							
147	IM-MOEA	Inverse modeling based multiobjective evolutionary algorithm		√		√	√				√								
148	IM-MOEA/D	Inverse modeling MOEA/D		√		√	√				√								
149	IMODE	Improved multi-operator differential evolution	√			√	√				√	√							
150	IMTCMO	Improved evolutionary multitasking-based CMOEA		√		√	√	√	√	√		√							
151	IMTCMO_BS	Improved evolutionary multitasking-based CMOEA with bidirectional sampling		√	√	√	√	√	√	√		√							
152	I-SIBEA	Interactive simple indicator-based evolutionary algorithm		√		√	√	√	√	√									
153	Izui	An aggregative gradient based multi-objective optimizer proposed by Izui et al.		√	√	√					√	√							
154	KAnD	Kriging-assisted AnD		√	√	√							√						
155	KLEA	Knowledge learning-based evolutionary algorithm		√		√	√		√		√	√			√				
156	KL-NSGA-II	Knowledge learning based NSGA-II		√		√	√	√	√	√		√				√			
157	KMA	Komodo mlipir algorithm	√			√	√				√	√							
158	KnEA	Knee point driven evolutionary algorithm			√	√	√	√	√	√		√							
159	K-RVEA	Surrogate-assisted RVEA		√	√	√	√						√						
160	KTA2	Kriging-assisted Two_Arch2		√	√	√	√						√						
161	KTS	Kriging-assisted evolutionary algorithm with two search modes		√	√		√					√	√						
162	L2SMEA	Linear subspace surrogate modeling assisted evolutionary algorithm	√			√							√						
163	LCMEA	Large-scale constrained multi-objective evolutionary algorithm		√		√					√	√							
164	LCSA	Linear combination-based search algorithm		√	√	√	√				√								
165	LDS-AF	Low-dimensional surrogate aggregation function		√		√	√				√		√						
166	LERD	Large-scale evolutionary algorithm with reformulated decision variable analysis		√	√	√					√								

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
167	LMEA	Evolutionary algorithm for large-scale many-objective optimization		√	√	√	√				√								
168	LMOCSO	Large-scale multi-objective competitive swarm optimization algorithm		√	√	√	√				√	√							
169	LMOEA-DS	Large-scale evolutionary multi-objective optimization assisted by directed sampling		√		√	√				√								
170	LMPFE	Evolutionary algorithm with local model based Pareto front estimation		√	√	√	√	√	√	√									
171	LRMOEA	Large-scale robust multi-objective evolutionary algorithm		√		√			√		√	√			√				√
172	LSMOF	Large-scale multi-objective optimization framework with NSGA-II		√		√	√				√								
173	MaOEA-CSS	Many-objective evolutionary algorithms based on coordinated selection		√	√	√	√	√	√	√									
174	MaOEA-DDFC	Many-objective evolutionary algorithm based on directional diversity and favorable convergence		√	√	√	√	√	√	√									
175	MaOEA-HAP	Hyper-curvature balanced indicator and adaptive phase exploration co-driven many-objective evolutionary algorithm		√	√	√	√	√	√	√									
176	MaOEA/IGD	IGD based many-objective evolutionary algorithm			√	√	√	√	√	√									
177	MaOEA/IT	Many-objective evolutionary algorithms based on an independent two-stage		√	√	√	√					√							
178	MaOEA-R&D	Many-objective evolutionary algorithm based on objective space reduction			√	√	√	√	√	√									
179	MaOTSPP	MaOEA based on two-stage probabilistic penalty		√	√	√						√	√						
180	MASSR-CMODRL	Constrained multi-objective optimization with MASSR states		√		√						√							
181	MASSR-MADE	Multi-agent differential evolution with MASSR states	√			√													
182	MASSR-MAMOEA/D	Multi-agent self-supervised state representation with MOEA/D		√		√													
183	MASSR-MARL-AC-S	Sparse multi-objective optimization with MASSR states		√		√			√		√				√				
184	MCCMO	Multi-population coevolutionary constrained multi-objective optimization		√		√	√	√	√	√		√							
185	MCEA/D	Multiple classifiers-assisted evolutionary algorithm based on decomposition		√	√	√	√						√						
186	MFEA	Multifactorial evolutionary algorithm	√			√	√	√	√	√	√						√		
187	MFEA-II	Multifactorial evolutionary algorithm II	√			√	√	√	√	√	√						√		
188	MFFS	Multiform feature selection		√					√										
189	MFO-SPEA2	Multiform optimization framework based on SPEA2		√		√	√	√	√	√		√							
190	MGCEA	Multi-granularity clustering based evolutionary algorithm		√		√			√		√	√			√				
191	MGO	Mountain gazelle optimizer	√			√	√				√	√							
192	MGSAEA	Multigranularity surrogate-assisted constrained evolutionary algorithm		√		√						√	√						
193	MiSACO	Multi surrogate-assisted ant colony optimization	√			√	√						√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
194	MMEA-ARM	Adaptive resource management based MMEA		√		√	√		√		√			√	√				
195	MMEAPSL	Multimodal multi-objective evolutionary algorithm assisted by Pareto set learning		√		√	√	√	√	√				√					
196	MMEA-WI	Weighted indicator-based evolutionary algorithm for multimodal multi-objective optimization		√		√	√							√					
197	MMOEA-BH	Multimodal MOEA with block optimization and hybrid clustering		√		√	√	√	√	√				√					
198	MMOPSO	MOPSO with multiple search strategies		√		√	√												
199	MO_Ring_PSO_SCD	Multiobjective PSO using ring topology and special crowding distance		√		√	√							√					
200	MOBCA	Multi-objective besiege and conquer algorithm		√		√	√												
201	MOCell	Cellular genetic algorithm		√		√	√	√	√	√		√							
202	MOCGDE	Multi-objective conjugate gradient and differential evolution algorithm		√	√	√					√	√							
203	MO-CMA	Multi-objective covariance matrix adaptation evolution strategy		√		√	√												
204	MOEA/CKF	Multi-objective evolutionary algorithm based on cross-scale knowledge fusion		√		√			√		√	√			√				
205	MOEA/D	Multiobjective evolutionary algorithm based on decomposition		√	√	√	√	√	√	√									
206	MOEA/D-2WA	MOEA/D with two-type weight vector adjustments		√	√	√	√	√	√	√		√							
207	MOEA/D-AWA	MOEA/D with adaptive weight adjustment		√	√	√	√	√	√	√									
208	MOEA/D-CMA	MOEA/D with covariance matrix adaptation evolution strategy		√	√	√	√												
209	MOEA/D-CMT	MOEA/D with competitive multitasking		√		√						√							
210	MOEA/DD	Many-objective evolutionary algorithm based on dominance and decomposition		√	√	√	√	√	√	√		√							
211	MOEA/D-DAE	MOEA/D with detect-and-escape strategy		√		√	√	√	√	√		√							
212	MOEA/D-DCWV	MOEA/D with distribution control of weight vector set		√	√	√	√	√	√	√									
213	MOEA/D-DE	MOEA/D based on differential evolution		√	√	√	√												
214	MOEA/D-DQN	MOEA/D based on deep Q-network		√	√	√	√												
215	MOEA/D-DRA	MOEA/D with dynamical resource allocation		√	√	√	√												
216	MOEA/D-DU	MOEA/D with a distance based updating strategy		√	√	√	√	√	√	√									
217	MOEA/D-DYTS	MOEA/D with dynamic Thompson sampling		√	√	√	√												
218	MOEA/D-EGO	MOEA/D with efficient global optimization		√		√	√						√						
219	MOEA/D-EGO-CEI	MOEA/D-EGO with CEI		√	√	√						√	√						
220	MOEA/D-FRRMAB	MOEA/D with fitness-rate-rank-based multiarmed bandit		√	√	√	√												
221	MOEA/D-M2M	MOEA/D based on MOP to MOP		√		√	√												
222	MOEA/D-MRDL	MOEA/D with maximum relative diversity loss		√		√	√												

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
223	MOEA/D-PaS	MOEA/D with Pareto adaptive scalarizing approximation		√	√	√	√												
224	MOEA/D-PFE	MOEA/D with Pareto front estimation		√	√	√	√	√	√	√									
225	MOEA/D-STM	MOEA/D with stable matching		√	√	√	√												
226	MOEA/D-UR	MOEA/D with update when required		√	√	√	√	√	√	√									
227	MOEA/D-URAW	MOEA/D with uniform randomly adaptive weights		√	√	√	√	√	√	√									
228	MOEA/DVA	Multi-objective evolutionary algorithm based on decision variable		√		√	√				√								
229	MOEA/D-VOV	MOEA/D with virtual objective vectors		√	√	√	√	√	√	√									
230	MOEA-IB	MOEA with information bottleneck		√	√	√					√								
231	MOEA/IGD-NS	Multi-objective evolutionary algorithm based on an enhanced IGD		√		√	√	√	√	√									
232	MOEA-NZD	Multi-objective evolutionary algorithm with nonzero detection		√	√	√					√	√			√				
233	MOEA-PC	Multiobjective evolutionary algorithm based on polar coordinates		√		√	√												
234	MOEA/PSL	Multi-objective evolutionary algorithm based on Pareto optimal subspace		√		√	√		√		√	√			√				
235	MOEA-RE	Multi-objective evolutionary algorithm with robustness enhancement		√		√	√	√	√	√									√
236	MOEA-SNN	Multi-objective evolutionary algorithm based on spiking neural networks		√		√			√		√				√				
237	MO-EGS	Multi-objective evolutionary gradient search		√		√					√								
238	MO-L2SMEA	Multi-objective linear subspace surrogate modeling assisted evolutionary algorithm		√		√					√		√						
239	MOMBI-II	Many objective metaheuristic based on the R2 indicator II		√	√	√	√	√	√	√									
240	MO-MFEA	Multi-objective multifactorial evolutionary algorithm		√		√	√	√	√	√		√					√		
241	MO-MFEA-II	Multi-objective multifactorial evolutionary algorithm II		√		√	√	√	√	√		√					√		
242	MOMFEA-SADE	Multi-objective multifactorial evolutionary algorithm with subspace alignment and adaptive differential evolution		√		√	√	√	√	√		√					√		
243	MONAS	Multi-objective neural architecture search		√		√	√	√	√	√				√					
244	MOPSO	Multi-objective particle swarm optimization		√		√	√												
245	MOPSO-CD	MOPSO with crowding distance		√		√	√												
246	MOSD	Multiobjective steepest descent		√		√					√	√							
247	M-PAES	Memetic algorithm with Pareto archived evolution strategy		√		√	√												
248	MP-MMEA	Multi-population multi-modal multi-objective evolutionary algorithm		√		√	√				√			√	√				
249	MPSO/D	Multi-objective particle swarm optimization algorithm based on decomposition		√	√	√	√												
250	MSCEA	Multi-stage constrained multi-objective evolutionary algorithm		√		√	√	√	√	√		√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
251	MSCMO	Multi-stage constrained multi-objective evolutionary algorithm		√		√	√	√	√	√		√							
252	MSEA	Multi-stage multi-objective evolutionary algorithm		√		√	√	√	√	√									
253	MSKEA	Multi-stage knowledge-guided evolutionary algorithm		√		√	√		√		√	√			√				
254	MSOPS-II	Multiple single objective Pareto sampling II		√	√	√	√					√							
255	MTCMO	Multitasking constrained multi-objective optimization		√		√	√	√	√	√		√							
256	MTDE-MKTA	Multitasking differential evolution with multiple knowledge types and transfer adaptation		√		√	√	√	√	√		√					√		
257	MTEA/D-DN	Multiobjective multitask evolutionary algorithm based on decomposition with dual neighborhoods		√		√	√	√	√	√		√					√		
258	MTS	Multiple trajectory search		√		√	√												
259	MultiObjective EGO	Multi-objective efficient global optimization		√		√	√					√	√						
260	MVPA	Most valuable player algorithm	√			√	√				√	√							
261	MyO-DEMR	Many-objective differential evolution with mutation restriction		√	√	√	√												
262	NBLEA	Nested bilevel evolutionary algorithm		√		√						√						√	
263	NelderMead	The Nelder-Mead algorithm	√			√													
264	NMPSO	Novel multi-objective particle swarm optimization		√	√	√	√												
265	NNDREA-MO	Evolutionary algorithm with neural network-based dimensionality reduction (multi-objective)		√					√		√	√			√				
266	NNDREA-SO	Evolutionary algorithm with neural network-based dimensionality reduction (single-objective)		√					√		√	√			√				
267	NNIA	Nondominated neighbor immune algorithm		√		√	√	√	√	√									
268	NRV-MOEA	Adaptive normal reference vector-based multi- and many-objective evolutionary algorithm		√	√	√	√	√	√	√									
269	NSBiDiCo	Non-dominated sorting bidirectional differential coevolution algorithm		√		√	√	√	√	√		√							
270	NSGA-II	Nondominated sorting genetic algorithm II		√		√	√	√	√	√		√							
271	NSGA-II+ARSBX	NSGA-II with adaptive rotation based simulated binary crossover		√		√	√					√							
272	NSGA-II-conflict	NSGA-II with conflict-based partitioning strategy			√	√	√	√	√	√									
273	NSGA-II-DTI	NSGA-II of Deb's type I robust version		√		√	√	√	√	√		√							√
274	NSGA-III	Nondominated sorting genetic algorithm III		√	√	√	√	√	√	√		√							
275	NSGAIII-EHVI	NSGA-III with expected hypervolume improvement		√	√	√							√						
276	NSGA-II/SDR	NSGA-II with strengthened dominance relation			√	√	√	√	√	√									
277	NSLS	Multiobjective optimization framework based on nondominated sorting and local search		√		√	√												
278	NUCEA	Non-uniform clustering based evolutionary algorithm		√		√			√		√	√			√				
279	OFA	Optimal foraging algorithm	√			√	√				√	√							
280	one-by-one EA	Many-objective evolutionary algorithm using a one-by-one selection		√	√	√	√	√	√	√									

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
281	OSP-NSDE	Non-dominated sorting differential evolution with prediction in the objective space		√		√	√												
282	ParEGO	Efficient global optimization for Pareto optimization		√		√	√						√						
283	PB-NSGA-III	NSGA-III based on Pareto based bi-indicator infill sampling criterion		√	√	√	√						√						
284	PB-RVEA	RVEA based on Pareto based bi-indicator infill sampling criterion		√	√	√	√						√						
285	PC-SAEA	Pairwise comparison based surrogate-assisted evolutionary algorithm		√	√								√						
286	PEA	Pareto-based Kriging-assisted constrained multiobjective evolutionary algorithm		√		√	√					√	√						
287	PEAplus	Pareto-based Kriging-assisted constrained multiobjective evolutionary algorithm plus		√		√	√					√	√						
288	PeEA	Pareto front shape estimation based evolutionary algorithm		√	√	√	√	√	√	√									
289	PESA-II	Pareto envelope-based selection algorithm II		√		√	√	√	√	√									
290	PICEA-g	Preference-inspired coevolutionary algorithm with goals		√	√	√	√	√	√	√									
291	PIEA	Performance indicator-based evolutionary algorithm		√	√	√							√						
292	PIMD	Probability and mapping crowding distance		√	√	√							√						
293	PM-MOEA	Pattern mining based multi-objective evolutionary algorithm		√		√	√		√		√	√			√				
294	POCEA	Paired offspring generation based constrained evolutionary algorithm		√		√	√				√	√							
295	PPS	Push and pull search algorithm		√	√	√	√					√							
296	PRCEA	Promising region-guided large-scale constrained MOEA		√	√	√	√				√	√							
297	PRDH	Problem reformulation and duplication handling		√					√										
298	PREA	Promising-region based EMO algorithm		√	√	√	√	√	√	√									
299	PSO	Particle swarm optimization	√			√	√				√	√							
300	REMO	Expensive multiobjective optimization by relation learning and prediction		√	√	√							√						
301	RGA-M1-2	Real-coded genetic algorithm with framework M1-2		√		√						√	√						
302	RGA-M2-2	Real-coded genetic algorithm with framework M2-2		√		√						√	√						
303	RM-MEDA	Regularity model-based multiobjective estimation of distribution		√		√	√												
304	RMOEA/DVA	Robust multi-objective evolutionary algorithm with decision variable assortment		√		√	√												√
305	RMSProp	Root mean square propagation	√			√					√								
306	r-NSGA-II	r-dominance based NSGA-II		√		√	√	√	√	√									
307	RPD-NSGA-II	Reference point dominance-based NSGA-II		√	√	√	√	√	√	√									
308	RPEA	Reference points-based evolutionary algorithm			√	√	√	√	√	√									
309	RSEA	Radial space division based evolutionary algorithm		√	√	√	√	√	√	√									

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
310	RVEA	Reference vector guided evolutionary algorithm		√	√	√	√	√	√	√		√							
311	RVEAa	RVEA embedded with the reference vector regeneration strategy			√	√	√	√	√	√									
312	RVEA-iGNG	RVEA based on improved growing neural gas		√	√	√	√	√	√	√									
313	S3-CMA-ES	Scalable small subpopulations based covariance matrix adaptation		√	√	√	√				√								
314	SA	Simulated annealing	√			√	√				√	√							
315	SACC-EAM-II	Surrogate-assisted cooperative co-evolutionary algorithm of Minamo	√			√	√						√						
316	SACOSO	Surrogate-assisted cooperative swarm optimization	√			√	√				√		√						
317	SADE-AMSS	Surrogate-assisted differential evolution with adaptive multi-subspace search	√			√	√						√						
318	SADE-ATDSC	Surrogate-assisted differential evolution with adaptation of training data selection criterion	√			√	√						√						
319	SADE-Sammon	Sammon mapping assisted differential evolution	√			√	√						√						
320	SAMOEA-TL2M	Surrogate-assisted multiobjective evolutionary algorithm based on two-level model management		√	√	√	√						√						
321	SAMSO	Multiswarm-assisted expensive optimization	√			√	√				√		√						
322	SAPO	Surrogate-assisted partial optimization	√			√	√					√	√						
323	S-CDAS	Self-controlling dominance area of solutions			√	√	√	√	√	√									
324	SCEA	Sparsity clustering basec evolutionary algorithm		√		√			√		√	√			√				
325	SCMOEA	Sparse constrained multi-objective evolutionary algorithm		√		√	√		√		√	√			√				
326	SD	Steepest descent	√			√					√								
327	S-ECSO	Enhanced competitive swarm optimizer for sparse optimization		√		√					√				√				
328	SFA-DE	Scalarization function approximation based differential evolution algorithm		√	√	√	√						√						
329	SFA-MFDE	Scalarization function approximation based multifactorial differential evolution algorithm		√	√	√	√						√						
330	SGEA	Steady-state and generational evolutionary algorithm		√		√	√	√	√	√		√				√			
331	SGECF	Sparsity-guided elitism co-evolutionary framework		√		√			√		√	√			√				
332	SHADE	Success-history based adaptive differential evolution	√			√	√				√	√							
333	SIBEA	Simple indicator-based evolutionary algorithm		√		√	√	√	√	√									
334	SIBEA-kEMOSS	SIBEA with minimum objective subset of size k with minimum error			√	√	√	√	√	√									
335	SLMEA	Super-large-scale multi-objective evolutionary algorithm		√		√	√		√		√	√			√				
336	SMEA	Self-organizing multiobjective evolutionary algorithm		√		√	√												
337	SMOA	Supervised multi-objective optimization algorithm		√		√							√						
338	SMPSO	Speed-constrained multi-objective particle swarm optimization		√		√	√												

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
339	SMS-EGO	S metric selection based efficient global optimization		√		√	√						√						
340	SMS-EMOA	S metric selection based evolutionary multiobjective optimization		√		√	√	√	√	√									
341	S-NSGA-II	Sparse NSGA-II		√		√					√	√			√				
342	SparseEA	Evolutionary algorithm for sparse multi-objective optimization problems		√		√	√		√		√	√			√				
343	SparseEA2	Improved SparseEA		√		√	√		√		√	√			√				
344	SparseEMT	Sparse evolutionary multitasking		√		√	√		√		√	√			√				
345	SPEA2	Strength Pareto evolutionary algorithm 2		√		√	√	√	√	√									
346	SPEA2+SDE	SPEA2 with shift-based density estimation			√	√	√	√	√	√									
347	SPEA/R	Strength Pareto evolutionary algorithm based on reference direction		√	√	√	√	√	√	√									
348	SQP	Sequential quadratic programming	√			√					√	√							
349	SRA	Stochastic ranking algorithm			√	√	√	√	√	√									
350	SSCEA	Subspace segmentation based co-evolutionary algorithm		√	√	√	√												
351	SSDE	Self-organized surrogate-assisted differential evolution		√	√	√	√					√	√						
352	SSIO-RL	Search space independent operator based deep reinforcement learning	√			√	√				√	√							
353	SVR-NSGA-II	Support vector regression based NSGA-II		√		√	√	√	√	√		√				√			
354	t-DEA	theta-dominance based evolutionary algorithm		√	√	√	√	√	√	√									
355	tDEA-CPBI	Theta-dominance based evolutionary algorithm with CPBI		√	√	√	√	√	√	√		√							
356	TEA	Two-phase evolutionary algorithm		√	√	√	√					√	√						
357	TELSO	Two-layer encoding learning swarm optimizer		√		√			√		√	√			√				
358	TiGE-2	Tri-Goal Evolution Framework for CMaOPs			√	√	√	√	√	√		√							
359	ToP	Two-phase framework with NSGA-II		√		√	√					√							
360	TPCMaO	Three-population based constrained many-objective co-evolutionary algorithm			√	√	√	√	√	√		√							
361	TriMOEA-TA&R	Multi-modal MOEA using two-archive and recombination strategies		√		√	√							√					
362	TS-NSGA-II	Two-stage NSGA-II		√	√	√	√	√	√	√									
363	TSPP	Two-stage probabilistic penalty		√	√	√						√	√						
364	TS-SparseEA	Two-stage SparseEA		√		√			√		√	√			√				
365	TSTI	Two-stage evolutionary algorithm with three indicators		√		√	√	√	√	√		√							
366	Two_Arch2	Two-archive algorithm 2		√	√	√	√	√	√	√									
367	UBCSO	Unbalanced and balanced competition strategy-assisted dual-swarm optimizer		√		√						√							
368	UICMO	Utilizing feasible non-dominated solution information for constrained multi-objective optimization		√		√						√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
369	URCMO	Utilizing the relationship between constrained and unconstrained Pareto fronts for constrained multi-objective optimization		√		√	√					√							
370	VaEA	Vector angle based evolutionary algorithm		√	√	√	√	√	√	√									
371	WASF-GA	Weighting achievement scalarizing function genetic algorithm		√		√	√	√	√	√									
372	WOA	Whale optimization algorithm	√			√	√				√	√							
373	WOF	Weighted optimization framework		√		√	√				√								
374	WV-MOEA-P	Weight vector based multi-objective optimization algorithm with preference		√		√	√												

VI. List of Problems

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
1	BBOB_F1	Sphere function	√			√							√						
2	BBOB_F2	Ellipsoidal function	√			√							√						
3	BBOB_F3	Rastrigin function	√			√							√						
4	BBOB_F4	Buche-Rastrigin function	√			√							√						
5	BBOB_F5	Linear slope	√			√							√						
6	BBOB_F6	Attractive sector function	√			√							√						
7	BBOB_F7	Step ellipsoidal function	√			√							√						
8	BBOB_F8	Rosenbrock function	√			√							√						
9	BBOB_F9	Rotated Rosenbrock function	√			√							√						
10	BBOB_F10	Rotated ellipsoidal function	√			√							√						
11	BBOB_F11	Discus function	√			√							√						
12	BBOB_F12	Bent cigar function	√			√							√						
13	BBOB_F13	Sharp ridge function	√			√							√						
14	BBOB_F14	Different powers function	√			√							√						
15	BBOB_F15	Rastrigin function	√			√							√						
16	BBOB_F16	Weierstrass function	√			√							√						
17	BBOB_F17	Schaffers F7 function	√			√							√						
18	BBOB_F18	Moderately ill-conditioned Schaffers F7 function	√			√							√						
19	BBOB_F19	Composite Griewank-Rosenbrock function F8F2	√			√							√						
20	BBOB_F20	Schwefel function	√			√							√						
21	BBOB_F21	Gallagher's Gaussian 101-me peaks function	√			√							√						
22	BBOB_F22	Gallagher's Gaussian 21-hi peaks function	√			√							√						
23	BBOB_F23	Katsuura function	√			√							√						
24	BBOB_F24	Lunacek bi-Rastrigin function	√			√							√						
25	BT1	Benchmark MOP with bias feature		√		√					√								
26	BT2	Benchmark MOP with bias feature		√		√					√								
27	BT3	Benchmark MOP with bias feature		√		√					√								
28	BT4	Benchmark MOP with bias feature		√		√					√								
29	BT5	Benchmark MOP with bias feature		√		√					√								
30	BT6	Benchmark MOP with bias feature		√		√					√								
31	BT7	Benchmark MOP with bias feature		√		√					√								
32	BT8	Benchmark MOP with bias feature		√		√					√								
33	BT9	Benchmark MOP with bias feature		√		√					√								

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
34	C10MOP1	Neural architecture search on CIFAR-10		√		√					√		√						
35	C10MOP2	Neural architecture search on CIFAR-10		√		√					√		√						
36	C10MOP3	Neural architecture search on CIFAR-10		√		√					√		√						
37	C10MOP4	Neural architecture search on CIFAR-10		√		√					√		√						
38	C10MOP5	Neural architecture search on CIFAR-10		√		√					√		√						
39	C10MOP6	Neural architecture search on CIFAR-10		√		√					√		√						
40	C10MOP7	Neural architecture search on CIFAR-10		√		√					√		√						
41	C10MOP8	Neural architecture search on CIFAR-10		√		√					√		√						
42	C10MOP9	Neural architecture search on CIFAR-10		√		√					√		√						
43	CEC2008_F1	Shifted sphere function	√			√					√		√						
44	CEC2008_F2	Shifted Schwefel's function	√			√					√		√						
45	CEC2008_F3	Shifted Rosenbrock's function	√			√					√		√						
46	CEC2008_F4	Shifted Rastrign's function	√			√					√		√						
47	CEC2008_F5	Shifted Griewank's function	√			√					√		√						
48	CEC2008_F6	Shifted Ackley's function	√			√					√		√						
49	CEC2008_F7	FastFractal 'DoubleDip' function	√			√					√		√						
50	CEC2010_F1	CEC'2010 constrained optimization benchmark problem	√			√						√							
51	CEC2010_F2	CEC'2010 constrained optimization benchmark problem	√			√						√							
52	CEC2010_F3	CEC'2010 constrained optimization benchmark problem	√			√						√							
53	CEC2010_F4	CEC'2010 constrained optimization benchmark problem	√			√						√							
54	CEC2010_F5	CEC'2010 constrained optimization benchmark problem	√			√						√							
55	CEC2010_F6	CEC'2010 constrained optimization benchmark problem	√			√						√							
56	CEC2010_F7	CEC'2010 constrained optimization benchmark problem	√			√						√							
57	CEC2010_F8	CEC'2010 constrained optimization benchmark problem	√			√						√							
58	CEC2010_F9	CEC'2010 constrained optimization benchmark problem	√			√						√							
59	CEC2010_F10	CEC'2010 constrained optimization benchmark problem	√			√						√							
60	CEC2010_F11	CEC'2010 constrained optimization benchmark problem	√			√						√							
61	CEC2010_F12	CEC'2010 constrained optimization benchmark problem	√			√						√							
62	CEC2010_F13	CEC'2010 constrained optimization benchmark problem	√			√						√							
63	CEC2010_F14	CEC'2010 constrained optimization benchmark problem	√			√						√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
64	CEC2010_F15	CEC'2010 constrained optimization benchmark problem	√			√						√							
65	CEC2010_F16	CEC'2010 constrained optimization benchmark problem	√			√						√							
66	CEC2010_F17	CEC'2010 constrained optimization benchmark problem	√			√						√							
67	CEC2010_F18	CEC'2010 constrained optimization benchmark problem	√			√						√							
68	CEC2013_F1	Shifted elliptic function	√			√					√								
69	CEC2013_F2	Shifted Rastrigin's function	√			√					√								
70	CEC2013_F3	Shifted Ackley's function	√			√					√								
71	CEC2013_F4	7-nonseparable, 1-separable shifted and rotated elliptic function	√			√					√								
72	CEC2013_F5	7-nonseparable, 1-separable shifted and rotated Rastrigin's function	√			√					√								
73	CEC2013_F6	7-nonseparable, 1-separable shifted and rotated Ackley's function	√			√					√								
74	CEC2013_F7	7-nonseparable, 1-separable shifted and rotated Schwefel's function	√			√					√								
75	CEC2013_F8	20-nonseparable shifted and rotated elliptic function	√			√					√								
76	CEC2013_F9	20-nonseparable shifted and rotated Rastrigin's function	√			√					√								
77	CEC2013_F10	20-nonseparable shifted and rotated Rastrigin's function	√			√					√								
78	CEC2013_F11	20-nonseparable shifted and rotated Schwefel's function	√			√					√								
79	CEC2013_F12	Shifted Rosenbrock's function	√			√					√								
80	CEC2013_F13	Shifted Schwefel's function with conforming overlapping subcomponents	√			√					√								
81	CEC2013_F14	Shifted Schwefel's function with conflicting overlapping subcomponents	√			√					√								
82	CEC2013_F15	Shifted Schwefel's function	√			√					√								
83	CEC2017_F1	CEC'2017 constrained optimization benchmark problem	√			√						√							
84	CEC2017_F2	CEC'2017 constrained optimization benchmark problem	√			√						√							
85	CEC2017_F3	CEC'2017 constrained optimization benchmark problem	√			√						√							
86	CEC2017_F4	CEC'2017 constrained optimization benchmark problem	√			√						√							
87	CEC2017_F5	CEC'2017 constrained optimization benchmark problem	√			√						√							
88	CEC2017_F6	CEC'2017 constrained optimization benchmark problem	√			√						√							
89	CEC2017_F7	CEC'2017 constrained optimization benchmark problem	√			√						√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
90	CEC2017_F8	CEC'2017 constrained optimization benchmark problem	√			√						√							
91	CEC2017_F9	CEC'2017 constrained optimization benchmark problem	√			√						√							
92	CEC2017_F10	CEC'2017 constrained optimization benchmark problem	√			√						√							
93	CEC2017_F11	CEC'2017 constrained optimization benchmark problem	√			√						√							
94	CEC2017_F12	CEC'2017 constrained optimization benchmark problem	√			√						√							
95	CEC2017_F13	CEC'2017 constrained optimization benchmark problem	√			√						√							
96	CEC2017_F14	CEC'2017 constrained optimization benchmark problem	√			√						√							
97	CEC2017_F15	CEC'2017 constrained optimization benchmark problem	√			√						√							
98	CEC2017_F16	CEC'2017 constrained optimization benchmark problem	√			√						√							
99	CEC2017_F17	CEC'2017 constrained optimization benchmark problem	√			√						√							
100	CEC2017_F18	CEC'2017 constrained optimization benchmark problem	√			√						√							
101	CEC2017_F19	CEC'2017 constrained optimization benchmark problem	√			√						√							
102	CEC2017_F20	CEC'2017 constrained optimization benchmark problem	√			√						√							
103	CEC2017_F21	CEC'2017 constrained optimization benchmark problem	√			√						√							
104	CEC2017_F22	CEC'2017 constrained optimization benchmark problem	√			√						√							
105	CEC2017_F23	CEC'2017 constrained optimization benchmark problem	√			√						√							
106	CEC2017_F24	CEC'2017 constrained optimization benchmark problem	√			√						√							
107	CEC2017_F25	CEC'2017 constrained optimization benchmark problem	√			√						√							
108	CEC2017_F26	CEC'2017 constrained optimization benchmark problem	√			√						√							
109	CEC2017_F27	CEC'2017 constrained optimization benchmark problem	√			√						√							
110	CEC2017_F28	CEC'2017 constrained optimization benchmark problem	√			√						√							
111	CEC2020_F1	Bent cigar function	√			√													
112	CEC2020_F2	Shifted and rotated Schwefel's function	√			√													
113	CEC2020_F3	Shifted and rotated Lunacek bi-Rastrigin function	√			√													
114	CEC2020_F4	Expanded Rosenbrock's plus Griewangk's function	√			√													

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
115	CEC2020_F5	Hybrid function 1	√			√													
116	CEC2020_F6	Hybrid function 2	√			√													
117	CEC2020_F7	Hybrid function 3	√			√													
118	CEC2020_F8	Composition function 1	√			√													
119	CEC2020_F9	Composition function 2	√			√													
120	CEC2020_F10	Composition function 3	√			√													
121	CF1	Constrained benchmark MOP		√		√					√	√							
122	CF2	Constrained benchmark MOP		√		√					√	√							
123	CF3	Constrained benchmark MOP		√		√					√	√							
124	CF4	Constrained benchmark MOP		√		√					√	√							
125	CF5	Constrained benchmark MOP		√		√					√	√							
126	CF6	Constrained benchmark MOP		√		√					√	√							
127	CF7	Constrained benchmark MOP		√		√					√	√							
128	CF8	Constrained benchmark MOP		√		√					√	√							
129	CF9	Constrained benchmark MOP		√		√					√	√							
130	CF10	Constrained benchmark MOP		√		√					√	√							
131	CI_HS	Multitasking problem (Griewank function + Rastrigin function)	√			√					√						√		
132	CI_LS	Multitasking problem (Ackley function + Schwefel function)	√			√					√						√		
133	CI_MS	Multitasking problem (Ackley function + Rastrigin function)	√			√					√						√		
134	CitySegMOP1	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
135	CitySegMOP2	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
136	CitySegMOP3	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
137	CitySegMOP4	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
138	CitySegMOP5	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
139	CitySegMOP6	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
140	CitySegMOP7	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
141	CitySegMOP8	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
142	CitySegMOP9	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
143	CitySegMOP10	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
144	CitySegMOP11	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
145	CitySegMOP12	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
146	CitySegMOP13	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
147	CitySegMOP14	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
148	CitySegMOP15	Neural architecture search on Cityscape segmentation datasets		√		√					√		√						
149	Community Detection	The community detection problem with label based encoding	√					√			√		√						
150	DAS-CMOP1	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
151	DAS-CMOP2	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
152	DAS-CMOP3	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
153	DAS-CMOP4	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
154	DAS-CMOP5	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
155	DAS-CMOP6	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
156	DAS-CMOP7	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
157	DAS-CMOP8	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
158	DAS-CMOP9	Difficulty-adjustable and scalable constrained benchmark MOP		√		√					√	√							
159	DOC1	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
160	DOC2	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
161	DOC3	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
162	DOC4	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
163	DOC5	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
164	DOC6	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
165	DOC7	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
166	DOC8	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
167	DOC9	Benchmark MOP with constraints in decision and objective spaces		√		√						√							
168	DSMOP1	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
169	DSMOP2	Dynamic sparse multi-objective optimization		√	√	√					√				√	√			

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
		problem																	
170	DSMOP3	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
171	DSMOP4	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
172	DSMOP5	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
173	DSMOP6	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
174	DSMOP7	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
175	DSMOP8	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
176	DSMOP9	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
177	DSMOP10	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
178	DSMOP11	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
179	DSMOP12	Dynamic sparse multi-objective optimization problem		√	√	√					√				√	√			
180	DTLZ1	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√		√						
181	DTLZ2	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√		√						
182	DTLZ3	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√		√						
183	DTLZ4	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√		√						
184	DTLZ5	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√		√						
185	DTLZ6	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√		√						
186	DTLZ7	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√		√						
187	DTLZ8	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√	√	√						
188	DTLZ9	Benchmark MOP proposed by Deb, Thiele, Laumanns, and Zitzler		√	√	√					√	√	√						
189	CDTLZ2	Convex DTLZ2		√	√	√					√		√						
190	IDTLZ1	Inverted DTLZ1		√	√	√					√		√						
191	IDTLZ2	Inverted DTLZ2		√	√	√					√		√						
192	IDTLZ3	Inverted DTLZ3		√	√	√					√		√						
193	IDTLZ4	Inverted DTLZ4		√	√	√					√		√						
194	Minus-DTLZ1	Minus DTLZ1		√	√	√					√		√						
195	Minus-DTLZ2	Minus DTLZ2		√	√	√					√		√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
196	Minus-DTLZ3	Minus DTLZ3		√	√	√					√		√						
197	Minus-DTLZ4	Minus DTLZ4		√	√	√					√		√						
198	Minus-DTLZ5	Minus DTLZ5		√	√	√					√		√						
199	Minus-DTLZ6	Minus DTLZ6		√	√	√					√		√						
200	Minus-DTLZ7	Minus DTLZ7		√	√	√					√		√						
201	Minus-IDTLZ1	Minus IDTLZ1		√	√	√					√		√						
202	Minus-IDTLZ2	Minus IDTLZ2		√	√	√					√		√						
203	Minus-IDTLZ3	Minus IDTLZ3		√	√	√					√		√						
204	Minus-IDTLZ4	Minus IDTLZ4		√	√	√					√		√						
205	SDTLZ1	Scaled DTLZ1		√	√	√					√		√						
206	SDTLZ2	Scaled DTLZ2		√	√	√					√		√						
207	C1-DTLZ1	Constrained DTLZ1		√	√	√					√	√	√						
208	C1-DTLZ3	Constrained DTLZ3		√	√	√					√	√	√						
209	C2-DTLZ2	Constrained DTLZ2		√	√	√					√	√	√						
210	C3-DTLZ4	Constrained DTLZ4		√	√	√					√	√	√						
211	DC1-DTLZ1	DTLZ1 with constrains in decision space		√	√	√					√	√	√						
212	DC1-DTLZ3	DTLZ3 with constrains in decision space		√	√	√					√	√	√						
213	DC2-DTLZ1	DTLZ1 with constrains in decision space		√	√	√					√	√	√						
214	DC2-DTLZ3	DTLZ3 with constrains in decision space		√	√	√					√	√	√						
215	DC3-DTLZ1	DTLZ1 with constrains in decision space		√	√	√					√	√	√						
216	DC3-DTLZ3	DTLZ3 with constrains in decision space		√	√	√					√	√	√						
217	EOPCCV_F1	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
218	EOPCCV_F2	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
219	EOPCCV_F3	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
220	EOPCCV_F4	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
221	EOPCCV_F5	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
222	EOPCCV_F6	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
223	EOPCCV_F7	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
224	EOPCCV_F8	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
225	EOPCCV_F9	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
226	EOPCCV_F10	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
227	EOPCCV_F11	Expensive optimization problems with continuous and categorical variables	√			√		√					√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
228	EOPCCV_F12	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
229	EOPCCV_F13	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
230	EOPCCV_F14	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
231	EOPCCV_F15	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
232	EOPCCV_F16	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
233	EOPCCV_F17	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
234	EOPCCV_F18	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
235	EOPCCV_F19	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
236	EOPCCV_F20	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
237	EOPCCV_F21	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
238	EOPCCV_F22	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
239	EOPCCV_F23	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
240	EOPCCV_F24	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
241	EOPCCV_F25	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
242	EOPCCV_F26	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
243	EOPCCV_F27	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
244	EOPCCV_F28	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
245	EOPCCV_F29	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
246	EOPCCV_F30	Expensive optimization problems with continuous and categorical variables	√			√		√					√						
247	FCP1	Benchmark constrained MOP proposed by Yuan		√		√						√							
248	FCP2	Benchmark constrained MOP proposed by Yuan		√		√						√							
249	FCP3	Benchmark constrained MOP proposed by Yuan		√		√						√							
250	FCP4	Benchmark constrained MOP proposed by Yuan		√		√						√							
251	FCP5	Benchmark constrained MOP proposed by Yuan		√		√						√							
252	FDA1	Benchmark dynamic MOP proposed by Farina, Deb, and Amato		√		√					√					√			
253	FDA2	Benchmark dynamic MOP proposed by Farina, Deb, and Amato		√		√					√					√			

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
254	FDA3	Benchmark dynamic MOP proposed by Farina, Deb, and Amato		√		√					√					√			
255	FDA4	Benchmark dynamic MOP proposed by Farina, Deb, and Amato		√		√					√					√			
256	FDA5	Benchmark dynamic MOP proposed by Farina, Deb, and Amato		√		√					√					√			
257	GLSMOP1	General large-scale benchmark MOP		√	√	√					√		√						
258	GLSMOP2	General large-scale benchmark MOP		√	√	√					√		√						
259	GLSMOP3	General large-scale benchmark MOP		√	√	√					√		√						
260	GLSMOP4	General large-scale benchmark MOP		√	√	√					√		√						
261	GLSMOP5	General large-scale benchmark MOP		√	√	√					√		√						
262	GLSMOP6	General large-scale benchmark MOP		√	√	√					√		√						
263	GLSMOP7	General large-scale benchmark MOP		√	√	√					√		√						
264	GLSMOP8	General large-scale benchmark MOP		√	√	√					√		√						
265	GLSMOP9	General large-scale benchmark MOP		√	√	√					√		√						
266	HYL1	Multi-modal multi-objective benchmark problem proposed by Hisao, Yiming, and Lie Meng		√		√								√					
267	HYL2	Multi-modal multi-objective benchmark problem proposed by Hisao, Yiming, and Lie Meng		√		√								√					
268	HYL3	Multi-modal multi-objective benchmark problem proposed by Hisao, Yiming, and Lie Meng		√		√								√					
269	HYL4	Multi-modal multi-objective benchmark problem proposed by Hisao, Yiming, and Lie Meng		√		√								√					
270	HYL5	Multi-modal multi-objective benchmark problem proposed by Hisao, Yiming, and Lie Meng		√		√								√					
271	IMMOEA_F1	Benchmark MOP for testing IM-MOEA		√		√					√								
272	IMMOEA_F2	Benchmark MOP for testing IM-MOEA		√		√					√								
273	IMMOEA_F3	Benchmark MOP for testing IM-MOEA		√		√					√								
274	IMMOEA_F4	Benchmark MOP for testing IM-MOEA		√		√					√								
275	IMMOEA_F5	Benchmark MOP for testing IM-MOEA		√		√					√								
276	IMMOEA_F6	Benchmark MOP for testing IM-MOEA		√		√					√								
277	IMMOEA_F7	Benchmark MOP for testing IM-MOEA		√		√					√								
278	IMMOEA_F8	Benchmark MOP for testing IM-MOEA		√		√					√								
279	IMMOEA_F9	Benchmark MOP for testing IM-MOEA		√		√					√								
280	IMMOEA_F10	Benchmark MOP for testing IM-MOEA		√		√					√								
281	IMOP1	Benchmark MOP with irregular Pareto front		√		√							√						
282	IMOP2	Benchmark MOP with irregular Pareto front		√		√							√						
283	IMOP3	Benchmark MOP with irregular Pareto front		√		√							√						
284	IMOP4	Benchmark MOP with irregular Pareto front		√		√							√						
285	IMOP5	Benchmark MOP with irregular Pareto front		√		√							√						
286	IMOP6	Benchmark MOP with irregular Pareto front		√		√							√						
287	IMOP7	Benchmark MOP with irregular Pareto front		√		√							√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
288	IMOP8	Benchmark MOP with irregular Pareto front		√		√							√						
289	IN1KMOP1	Neural architecture search on ImageNet 1K		√		√					√		√						
290	IN1KMOP2	Neural architecture search on ImageNet 1K		√		√					√		√						
291	IN1KMOP3	Neural architecture search on ImageNet 1K		√		√					√		√						
292	IN1KMOP4	Neural architecture search on ImageNet 1K		√		√					√		√						
293	IN1KMOP5	Neural architecture search on ImageNet 1K		√		√					√		√						
294	IN1KMOP6	Neural architecture search on ImageNet 1K		√		√					√		√						
295	IN1KMOP7	Neural architecture search on ImageNet 1K		√		√					√		√						
296	IN1KMOP8	Neural architecture search on ImageNet 1K		√		√					√		√						
297	IN1KMOP9	Neural architecture search on ImageNet 1K		√		√					√		√						
298	Instance1	Multitasking multi-objective problem (ZDT4-R + ZDT4-G)		√		√					√						√		
299	Instance2	Multitasking multi-objective problem (ZDT4-RC + ZDT4-A)		√		√					√	√					√		
300	KP	The knapsack problem	√						√		√	√							
301	LIR-CMOP1	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
302	LIR-CMOP2	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
303	LIR-CMOP3	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
304	LIR-CMOP4	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
305	LIR-CMOP5	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
306	LIR-CMOP6	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
307	LIR-CMOP7	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
308	LIR-CMOP8	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
309	LIR-CMOP9	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
310	LIR-CMOP10	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
311	LIR-CMOP11	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
312	LIR-CMOP12	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
313	LIR-CMOP13	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
314	LIR-CMOP14	Constrained benchmark MOP with large infeasible regions		√		√					√	√							
315	LRMOP1	Large-scale robust multi-objective benchmark problem		√	√	√					√		√		√				√

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
316	LRMOP2	Large-scale robust multi-objective benchmark problem		√	√	√					√		√		√				√
317	LRMOP3	Large-scale robust multi-objective benchmark problem		√	√	√					√		√		√				√
318	LRMOP4	Large-scale robust multi-objective benchmark problem		√	√	√					√		√		√				√
319	LRMOP5	Large-scale robust multi-objective benchmark problem		√	√	√					√		√		√				√
320	LRMOP6	Large-scale robust multi-objective benchmark problem		√	√	√					√		√		√				√
321	LSCM1	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
322	LSCM2	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
323	LSCM3	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
324	LSCM4	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
325	LSCM5	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
326	LSCM6	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
327	LSCM7	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
328	LSCM8	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
329	LSCM9	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
330	LSCM10	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
331	LSCM11	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
332	LSCM12	Large-scale constrained multiobjective benchmark problem		√		√					√	√							
333	LSMOP1	Large-scale benchmark MOP		√	√	√					√								
334	LSMOP2	Large-scale benchmark MOP		√	√	√					√								
335	LSMOP3	Large-scale benchmark MOP		√	√	√					√								
336	LSMOP4	Large-scale benchmark MOP		√	√	√					√								
337	LSMOP5	Large-scale benchmark MOP		√	√	√					√								
338	LSMOP6	Large-scale benchmark MOP		√	√	√					√								
339	LSMOP7	Large-scale benchmark MOP		√	√	√					√								
340	LSMOP8	Large-scale benchmark MOP		√	√	√					√								
341	LSMOP9	Large-scale benchmark MOP		√	√	√					√								
342	MaF1	Inverted DTLZ1		√	√	√					√								
343	MaF2	DTLZ2BZ		√	√	√					√								

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
344	MaF3	Convex DTLZ3		√	√	√					√								
345	MaF4	Inverted and scaled DTLZ3		√	√	√					√								
346	MaF5	Scaled DTLZ4		√	√	√					√								
347	MaF6	DTLZ5IM		√	√	√					√								
348	MaF7	DTLZ7		√	√	√					√								
349	MaF8	MP-DMP		√	√	√													
350	MaF9	ML-DMP		√	√	√													
351	MaF10	WFG1		√	√	√					√								
352	MaF11	WFG2		√	√	√					√								
353	MaF12	WFG9		√	√	√					√								
354	MaF13	P7		√	√	√					√								
355	MaF14	LSMOP3		√	√	√					√								
356	MaF15	Inverted LSMOP8		√	√	√					√								
357	MaOPP_binary	Many-objective pathfinding problem based on binary encoding			√				√		√		√						
358	MaOPP_real	Many-objective pathfinding problem based on real encoding			√	√					√		√						
359	Mario	Play with Mario	√				√	√											
360	MaxCut	The max-cut problem	√						√		√								
361	MLDMP	The multi-line distance minimization problem		√	√	√													
362	MMF1	Multi-modal multi-objective test function		√		√								√					
363	MMF2	Multi-modal multi-objective test function		√		√								√					
364	MMF3	Multi-modal multi-objective test function		√		√								√					
365	MMF4	Multi-modal multi-objective test function		√		√								√					
366	MMF5	Multi-modal multi-objective test function		√		√								√					
367	MMF6	Multi-modal multi-objective test function		√		√								√					
368	MMF7	Multi-modal multi-objective test function		√		√								√					
369	MMF8	Multi-modal multi-objective test function		√		√								√					
370	MMMOP1	Multi-modal multi-objective optimization problem		√	√	√								√					
371	MMMOP2	Multi-modal multi-objective optimization problem		√	√	√								√					
372	MMMOP3	Multi-modal multi-objective optimization problem		√	√	√								√					
373	MMMOP4	Multi-modal multi-objective optimization problem		√	√	√								√					
374	MMMOP5	Multi-modal multi-objective optimization problem		√	√	√								√					
375	MMMOP6	Multi-modal multi-objective optimization problem		√	√	√								√					
376	MMOP_HS1	Large-scale sparse multitasking multi-objective optimization problem		√		√					√				√		√		
377	MMOP_HS2	Large-scale sparse multitasking multi-objective optimization problem		√		√					√				√		√		
378	MMOP_LS1	Large-scale sparse multitasking multi-objective optimization problem		√		√					√				√		√		

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
379	MMOP_LS2	Large-scale sparse multitasking multi-objective optimization problem		√		√					√				√		√		
380	MMOP_MS1	Large-scale sparse multitasking multi-objective optimization problem		√		√					√				√		√		
381	MMOP_MS2	Large-scale sparse multitasking multi-objective optimization problem		√		√					√				√		√		
382	MMOP_NS1	Large-scale sparse multitasking multi-objective optimization problem		√		√					√				√		√		
383	MMOP_NS2	Large-scale sparse multitasking multi-objective optimization problem		√		√					√				√		√		
384	MOEADDE_F1	Benchmark MOP for testing MOEA/D-DE		√		√					√								
385	MOEADDE_F2	Benchmark MOP for testing MOEA/D-DE		√		√					√								
386	MOEADDE_F3	Benchmark MOP for testing MOEA/D-DE		√		√					√								
387	MOEADDE_F4	Benchmark MOP for testing MOEA/D-DE		√		√					√								
388	MOEADDE_F5	Benchmark MOP for testing MOEA/D-DE		√		√					√								
389	MOEADDE_F6	Benchmark MOP for testing MOEA/D-DE		√		√					√								
390	MOEADDE_F7	Benchmark MOP for testing MOEA/D-DE		√		√					√								
391	MOEADDE_F8	Benchmark MOP for testing MOEA/D-DE		√		√					√								
392	MOEADDE_F9	Benchmark MOP for testing MOEA/D-DE		√		√					√								
393	MOEADM2M_F1	Benchmark MOP for testing MOEA/D-M2M		√		√					√								
394	MOEADM2M_F2	Benchmark MOP for testing MOEA/D-M2M		√		√					√								
395	MOEADM2M_F3	Benchmark MOP for testing MOEA/D-M2M		√		√					√								
396	MOEADM2M_F4	Benchmark MOP for testing MOEA/D-M2M		√		√					√								
397	MOEADM2M_F5	Benchmark MOP for testing MOEA/D-M2M		√		√					√								
398	MOEADM2M_F6	Benchmark MOP for testing MOEA/D-M2M		√		√					√								
399	MOEADM2M_F7	Benchmark MOP for testing MOEA/D-M2M		√		√					√								
400	MOKP	The multi-objective knapsack problem		√	√				√		√	√							
401	MONRP	The multi-objective next release problem		√					√		√								
402	MOTSP	The multi-objective traveling salesman problem		√	√					√	√								
403	MPDMP	The multi-point distance minimization problem		√	√	√													
404	mQAP	The multi-objective quadratic assignment problem		√	√					√	√								
405	MW1	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
406	MW2	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
407	MW3	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
408	MW4	Constrained benchmark MOP proposed by Ma and Wang		√	√	√					√	√							
409	MW5	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
410	MW6	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
411	MW7	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
412	MW8	Constrained benchmark MOP proposed by Ma and Wang		√	√	√					√	√							
413	MW9	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
414	MW10	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
415	MW11	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
416	MW12	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
417	MW13	Constrained benchmark MOP proposed by Ma and Wang		√		√					√	√							
418	MW14	Constrained benchmark MOP proposed by Ma and Wang		√	√	√					√	√							
419	NI_HS	Multitasking problem (Rosenbrock function + Rastrigin function)	√			√					√						√		
420	NI_MS	Multitasking problem (Griewank function + Weierstrass function)	√			√					√						√		
421	RMMEDA_F1	Benchmark MOP for testing RM-MEDA		√		√					√								
422	RMMEDA_F2	Benchmark MOP for testing RM-MEDA		√		√					√								
423	RMMEDA_F3	Benchmark MOP for testing RM-MEDA		√		√					√								
424	RMMEDA_F4	Benchmark MOP for testing RM-MEDA		√		√					√								
425	RMMEDA_F5	Benchmark MOP for testing RM-MEDA		√		√					√								
426	RMMEDA_F6	Benchmark MOP for testing RM-MEDA		√		√					√								
427	RMMEDA_F7	Benchmark MOP for testing RM-MEDA		√		√					√								
428	RMMEDA_F8	Benchmark MOP for testing RM-MEDA		√		√					√								
429	RMMEDA_F9	Benchmark MOP for testing RM-MEDA		√		√					√								
430	RMMEDA_F10	Benchmark MOP for testing RM-MEDA		√		√					√								
431	RWMOP1	Pressure vessal problem		√		√						√							
432	RWMOP2	Vibrating platform		√		√						√							
433	RWMOP3	Two bar truss design problem		√		√						√							
434	RWMOP4	Weldan beam design problem		√		√						√							
435	RWMOP5	Disc brake design problem		√		√						√							
436	RWMOP6	Speed reducer design problem		√		√						√							
437	RWMOP7	Gear train design problem		√		√						√							
438	RWMOP8	Car side impact design problem		√		√						√							
439	RWMOP9	Four bar plane truss		√		√						√							
440	RWMOP10	Two bar plane truss		√		√						√							
441	RWMOP11	Water resource management problem		√		√						√							
442	RWMOP12	Simply supported I-beam design		√		√						√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
443	RWMOP13	Gear box design		√		√						√							
444	RWMOP14	Multiple-disk clutch brake design problem		√		√						√							
445	RWMOP15	Spring design problem		√		√						√							
446	RWMOP16	Cantilever beam design problem		√		√						√							
447	RWMOP17	Bulk carriers design problem		√		√						√							
448	RWMOP18	Front rail design problem		√		√						√							
449	RWMOP19	Multi-product batch plant		√		√						√							
450	RWMOP20	Hydro-static thrust bearing design problem		√		√						√							
451	RWMOP21	Crash energy management for high-speed train		√		√						√							
452	RWMOP22	Haverly's pooling problem		√		√						√							
453	RWMOP23	Reactor network design		√		√						√							
454	RWMOP24	Heat exchanger network design		√		√						√							
455	RWMOP25	Process synthesis problem		√		√						√							
456	RWMOP26	Process sythesis and design problem		√		√						√							
457	RWMOP27	Process flow sheeting problem		√		√						√							
458	RWMOP28	Two reactor problem		√		√						√							
459	RWMOP29	Process synthesis problem		√		√						√							
460	RWMOP30	Synchronous pptimal pulse-width modulation of 3-level inverters		√		√						√							
461	RWMOP31	Synchronous pptimal pulse-width modulation of 5-level inverters		√		√						√							
462	RWMOP32	Synchronous pptimal pulse-width modulation of 7-level inverters		√		√						√							
463	RWMOP33	Synchronous pptimal pulse-width modulation of 9-level inverters		√		√						√							
464	RWMOP34	Synchronous pptimal pulse-width modulation of 11-level inverters		√		√						√							
465	RWMOP35	Synchronous pptimal pulse-width modulation of 13-level inverters		√		√						√							
466	RWMOP36	Optimal sizing of single phase distributed generation with reactive power support for phase balancing at main transformer/grid and active power loss		√		√						√							
467	RWMOP37	Optimal Sizing of Single Phase Distributed Generation with reactive power support for Phase Balancing at Main Transformer/Grid and reactive Power loss		√		√						√							
468	RWMOP38	Optimal sizing of single phase distributed generation with reactive power support for active and reactive power loss		√		√						√							
469	RWMOP39	Optimal sizing of single phase distributed generation with reactive power support for phase balancing at main transformer/grid and active and reactive power loss		√		√						√							
470	RWMOP40	Optimal power flow for minimizing active and reactive power loss		√		√						√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
471	RWMOP41	Optimal power flow for minimizing voltage deviation, active and reactive power loss		√		√						√							
472	RWMOP42	Optimal power flow for minimizing voltage deviation, and active power loss		√		√						√							
473	RWMOP43	Optimal power flow for minimizing fuel cost, and active power loss		√		√						√							
474	RWMOP44	Optimal power flow for minimizing fuel cost, active and reactive power loss		√		√						√							
475	RWMOP45	Optimal power flow for minimizing fuel cost, voltage deviation, and active power loss		√		√						√							
476	RWMOP46	Optimal power flow for minimizing fuel cost, voltage deviation, active and reactive power loss		√		√						√							
477	RWMOP47	Optimal droop setting for minimizing active and reactive power loss		√		√						√							
478	RWMOP48	Optimal droop setting for minimizing voltage deviation and active power loss		√		√						√							
479	RWMOP49	Optimal droop setting for minimizing voltage deviation, active, and reactive power loss		√		√						√							
480	RWMOP50	Power distribution system planning		√		√						√							
481	SCMOP1	Benchmark CMOP with sparse Pareto optimal solutions		√	√	√					√	√	√		√				
482	SCMOP2	Benchmark CMOP with sparse Pareto optimal solutions		√	√	√					√	√	√		√				
483	SCMOP3	Benchmark CMOP with sparse Pareto optimal solutions		√	√	√					√	√	√		√				
484	SCMOP4	Benchmark CMOP with sparse Pareto optimal solutions		√	√	√					√	√	√		√				
485	SCMOP5	Benchmark CMOP with sparse Pareto optimal solutions		√	√	√					√	√	√		√				
486	SCMOP6	Benchmark CMOP with sparse Pareto optimal solutions		√	√	√					√	√	√		√				
487	SCMOP7	Benchmark CMOP with sparse Pareto optimal solutions		√	√	√					√	√	√		√				
488	SCMOP8	Benchmark CMOP with sparse Pareto optimal solutions		√	√	√					√	√	√		√				
489	SDC1	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
490	SDC2	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
491	SDC3	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
492	SDC4	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
493	SDC5	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
494	SDC6	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
495	SDC7	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
496	SDC8	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
497	SDC9	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
498	SDC10	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
499	SDC11	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
500	SDC12	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
501	SDC13	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
502	SDC14	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
503	SDC15	Scalable high-dimensional decicsion constraint benchamrk		√		√						√							
504	SMD1	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√												√	
505	SMD2	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√												√	
506	SMD3	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√												√	
507	SMD4	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√												√	
508	SMD5	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√												√	
509	SMD6	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√												√	
510	SMD7	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√												√	
511	SMD8	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√												√	
512	SMD9	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√						√						√	
513	SMD10	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√						√						√	
514	SMD11	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√						√						√	
515	SMD12	Bilevel optimization problems proposed by Sinha, Malo, and Deb		√		√						√						√	
516	SO_ISCSO_2016	International student competition in structural optimization	√				√				√	√							
517	SO_ISCSO_2017	International student competition in structural optimization	√				√				√	√							
518	SO_ISCSO_2018	International student competition in structural optimization	√				√				√	√							
519	SO_ISCSO_2019	International student competition in structural optimization	√				√				√	√							
520	SO_ISCSO_2021	International student competition in structural	√				√				√	√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
		optimization																	
521	SO_ISCSO_2022	International student competition in structural optimization	√				√				√	√							
522	Sparse_CD	The community detection problem		√					√		√		√		√				
523	Sparse_CN	The critical node detection problem		√					√		√		√		√				
524	Sparse_FS	The feature selection problem		√					√		√		√		√				
525	Sparse_IS	The instance selection problem		√					√		√		√		√				
526	Sparse_KP	The sparse multi-objective knapsack problem		√	√				√		√								
527	Sparse_NN	The neural network training problem		√		√					√		√		√				
528	Sparse_PM	The pattern mining problem		√					√		√		√		√				
529	Sparse_PO	The portfolio optimization problem		√		√					√		√		√				
530	Sparse_SR	The sparse signal reconstruction problem		√		√					√		√		√				
531	SMMOP1	Sparse multi-modal multi-objective optimization problem		√	√	√					√			√	√				
532	SMMOP2	Sparse multi-modal multi-objective optimization problem		√	√	√					√			√	√				
533	SMMOP3	Sparse multi-modal multi-objective optimization problem		√	√	√					√			√	√				
534	SMMOP4	Sparse multi-modal multi-objective optimization problem		√	√	√					√			√	√				
535	SMMOP5	Sparse multi-modal multi-objective optimization problem		√	√	√					√			√	√				
536	SMMOP6	Sparse multi-modal multi-objective optimization problem		√	√	√					√			√	√				
537	SMMOP7	Sparse multi-modal multi-objective optimization problem		√	√	√					√			√	√				
538	SMMOP8	Sparse multi-modal multi-objective optimization problem		√	√	√					√			√	√				
539	SMOP1	Benchmark MOP with sparse Pareto optimal solutions		√	√	√					√		√		√				
540	SMOP2	Benchmark MOP with sparse Pareto optimal solutions		√	√	√					√		√		√				
541	SMOP3	Benchmark MOP with sparse Pareto optimal solutions		√	√	√					√		√		√				
542	SMOP4	Benchmark MOP with sparse Pareto optimal solutions		√	√	√					√		√		√				
543	SMOP5	Benchmark MOP with sparse Pareto optimal solutions		√	√	√					√		√		√				
544	SMOP6	Benchmark MOP with sparse Pareto optimal solutions		√	√	√					√		√		√				
545	SMOP7	Benchmark MOP with sparse Pareto optimal solutions		√	√	√					√		√		√				
546	SMOP8	Benchmark MOP with sparse Pareto optimal solutions		√	√	√					√		√		√				
547	SOP_F1	Sphere function	√			√							√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
548	SOP_F2	Schwefel's function 2.22	√			√							√						
549	SOP_F3	Schwefel's function 1.2	√			√							√						
550	SOP_F4	Schwefel's function 2.21	√			√							√						
551	SOP_F5	Generalized Rosenbrock's function	√			√							√						
552	SOP_F6	Step function	√			√							√						
553	SOP_F7	Quartic function with noise	√			√							√						
554	SOP_F8	Generalized Schwefel's function 2.26	√			√							√						
555	SOP_F9	Generalized Rastrigin's function	√			√							√						
556	SOP_F10	Ackley's function	√			√							√						
557	SOP_F11	Generalized Griewank's function	√			√							√						
558	SOP_F12	Generalized penalized function	√			√							√						
559	SOP_F13	Generalized penalized function	√			√							√						
560	SOP_F14	Shekel's foxholes function	√			√							√						
561	SOP_F15	Kowalik's function	√			√							√						
562	SOP_F16	Six-hump camel-back function	√			√							√						
563	SOP_F17	Branin function	√			√							√						
564	SOP_F18	Goldstein-price function	√			√							√						
565	SOP_F19	Hartman's family	√			√							√						
566	SOP_F20	Hartman's family	√			√							√						
567	SOP_F21	Shekel's family	√			√							√						
568	SOP_F22	Shekel's family	√			√							√						
569	SOP_F23	Shekel's family	√			√							√						
570	TP1	Test problem for robust multi-objective optimization		√		√					√								√
571	TP2	Test problem for robust multi-objective optimization		√		√					√								√
572	TP3	Test problem for robust multi-objective optimization		√		√					√								√
573	TP4	Test problem for robust multi-objective optimization		√		√					√								√
574	TP5	Test problem for robust multi-objective optimization		√		√					√								√
575	TP6	Test problem for robust multi-objective optimization		√		√					√								√
576	TP7	Test problem for robust multi-objective optimization		√		√					√								√
577	TP8	Test problem for robust multi-objective optimization		√		√					√								√
578	TP9	Test problem for robust multi-objective optimization		√		√					√								√
579	TP10	Test problem for robust multi-objective optimization		√		√					√	√							√
580	TREE1	The time-varying ratio error estimation problem		√		√					√	√	√						
581	TREE2	The time-varying ratio error estimation problem		√		√					√	√	√						
582	TREE3	The time-varying ratio error estimation problem		√		√					√	√	√						
583	TREE4	The time-varying ratio error estimation problem		√		√					√	√	√						
584	TREE5	The time-varying ratio error estimation problem		√		√					√	√	√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
585	TREE6	The time-varying ratio error estimation problem		√		√					√	√	√						
586	TSP	The traveling salesman problem	√							√	√								
587	UF1	Unconstrained benchmark MOP		√		√					√								
588	UF2	Unconstrained benchmark MOP		√		√					√								
589	UF3	Unconstrained benchmark MOP		√		√					√								
590	UF4	Unconstrained benchmark MOP		√		√					√								
591	UF5	Unconstrained benchmark MOP		√		√					√								
592	UF6	Unconstrained benchmark MOP		√		√					√								
593	UF7	Unconstrained benchmark MOP		√		√					√								
594	UF8	Unconstrained benchmark MOP		√		√					√								
595	UF9	Unconstrained benchmark MOP		√		√					√								
596	UF10	Unconstrained benchmark MOP		√		√					√								
597	VNT1	Benchmark MOP proposed by Viennet		√		√													
598	VNT2	Benchmark MOP proposed by Viennet		√		√													
599	VNT3	Benchmark MOP proposed by Viennet		√		√													
600	VNT4	Benchmark MOP proposed by Viennet		√		√						√							
601	WFG1	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
602	WFG2	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
603	WFG3	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
604	WFG4	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
605	WFG5	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
606	WFG6	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
607	WFG7	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
608	WFG8	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
609	WFG9	Benchmark MOP proposed by Walking Fish Group		√	√	√					√		√						
610	Minus-WFG1	Minus WFG1		√	√	√					√		√						
611	Minus-WFG2	Minus WFG2		√	√	√					√		√						
612	Minus-WFG3	Minus WFG3		√	√	√					√		√						
613	Minus-WFG4	Minus WFG4		√	√	√					√		√						
614	Minus-WFG5	Minus WFG5		√	√	√					√		√						
615	Minus-WFG6	Minus WFG6		√	√	√					√		√						
616	Minus-WFG7	Minus WFG7		√	√	√					√		√						
617	Minus-WFG8	Minus WFG8		√	√	√					√		√						
618	Minus-WFG9	Minus WFG9		√	√	√					√		√						
619	ZCAT1	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
620	ZCAT2	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
621	ZCA3	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
622	ZCA4	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
623	ZCA5	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
624	ZCAT6	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
625	ZCAT7	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
626	ZCAT8	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
627	ZCAT9	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
628	ZCAT10	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
629	ZCAT11	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
630	ZCAT12	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
631	ZCAT13	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
632	ZCAT14	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
633	ZCAT15	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
634	ZCAT16	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
635	ZCAT17	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
636	ZCAT18	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
637	ZCAT19	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
638	ZCAT20	Benchmark MOP proposed by Zapotecas, Coello, Aguirre, and Tanaka		√	√	√					√		√						
639	ZDT1	Benchmark MOP proposed by Zitzler, Deb, and Thiele		√		√					√		√						
640	ZDT2	Benchmark MOP proposed by Zitzler, Deb, and Thiele		√		√					√		√						
641	ZDT3	Benchmark MOP proposed by Zitzler, Deb, and Thiele		√		√					√		√						
642	ZDT4	Benchmark MOP proposed by Zitzler, Deb, and Thiele		√		√					√		√						
643	ZDT5	Benchmark MOP proposed by Zitzler, Deb, and Thiele		√					√		√		√						
644	ZDT6	Benchmark MOP proposed by Zitzler, Deb, and Thiele		√		√					√		√						
645	ZXH_CF1	Constrained benchmark MOP proposed by		√	√	√					√	√							

	Abbreviation	Full name	single	multi	many	real	integer	label	binary	permutation	large	constrained	expensive	multimodal	sparse	dynamic	multitask	bilevel	robust
		Zhou, Xiang, and He																	
646	ZXH_CF2	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
647	ZXH_CF3	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
648	ZXH_CF4	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
649	ZXH_CF5	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
650	ZXH_CF6	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
651	ZXH_CF7	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
652	ZXH_CF8	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
653	ZXH_CF9	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
654	ZXH_CF10	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
655	ZXH_CF11	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
656	ZXH_CF12	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
657	ZXH_CF13	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
658	ZXH_CF14	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
659	ZXH_CF15	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							
660	ZXH_CF16	Constrained benchmark MOP proposed by Zhou, Xiang, and He		√	√	√					√	√							